

IntentSpider - The Fluid Language Web for Textual Tension and Prediction

The Architecture, Valence Gated Extensions, a Gating System and a Evaluation Protocol with Debugging

Neksha DeSilva

Phillips Academy Andover

Research Paper



IntentSpider Webnet is dedicated to the memory of NASA's Apollo 11, and to the American heroes who, on July 20th, 1969, became the first conscious beings from the Planet Earth to set foot on another world.



Abstract

IntentSpider is an on device, privacy first, and privacy preserving predictive text engine, built and tested based on Standard Template Library in C++. This is a personal intent graph engine with dynamic edges in which every edge is typed by the accuracy of the typed words (or) array of characters without spaces, whether it reliably carries signal pattern towards what the users, in reality, go to select, rather than the time or how long it has been used. This decision was based on the analogy of the PageRank algorithm, used by Google Search Engine, seeded from several recent tokens, considered by how diversely each array's own reliable connections make more connections with other arrays (fanning algorithms), rather than with a direct search from a one token. The method is similar to how search engines rank webpages, it is an established technique, and we cover that part in detail. Those connections, which are labeled by their process, and the spread-weighted seed built on top of this, are novel subsystems, motivated by the weaver spider's web silk study.

IntentSpider core has one structural limitation, here we termed as the blindness to the valence, which implies that, it is unable to differentiate a selection or a next word that confirms what the user implied from the selection that closes the exchange with the other end(human, machine, or a processing interface) because neither the connection strength update nor updates on reliability considers the rest of the sentence that word came from. To address this, IntentSpider uses a signed, valence-gated extension, with a light, internal valence signal taken from WPM, or words per minute, via a simple algorithm and correction behavior based on previous research on typing errors and increased frontal theta activity in the human brain during keyboard interaction, rather than a default list of emotional words. This signal is also motivated by a finding from the orb spider's reflections. As an analogy, a web's

vibration frequency without a custom dictionary of words decides whether the IntentSpider approaches or withdraws. Even further, the separate gating mechanism, as declared here as necessity gating algorithm, built openly using a creative reconstruction of SCAMPER methods, to decide whether an array of characters should be added to the X number of the next words vector, which is independent of which word is chosen. Building on the valence signal, we add a second, independent signal, "hexarousal", which is a highly variable, intensity value, without a positive or negative direction, taken from how much a person's current typing speed (word count per minute) differs from their average speed.

In addition to this, IntentSpider includes six other engines, motivated by an attacking and struggle analogy of the Orb Spider, Wandering Spider, and the Rolling Isopod prey. This is a reinforcement that compounds across several live candidates at once. A radius of signals that grows under a consistent hexarousal and a sudden suppression event that returns a value for the shock that occurred. And it is separate from general, ordinary valence-based suppression algorithm. This stabilizing rule treats confirmed and unconfirmed excited states exchanges differently. IntentSpider also consists of an unsupervised discovery step that finds however many distinguishable excited states a person's typing actually shows per each user and a nonlinear combination of behavioral features in place of a single hexarousal number as well.

As a separate component, we add a graph theory based, live spectral embedding of the same graph per each user. turning each typed sentence into a path through a two dimensional working plane, and the delta between two sentences(a sentence implies an array of space separated characters) into a measurable object, as a shape rather than a single number. This feature does not change the system behavior and is not an input to the ranking process, reinforcement engine, or the gating process. This can be interpreted as a debugging system as well.

Keywords: ML, PageRank, Graph Diffusion, Entropy Weighted Seeding, Biometrics, Dynamic Graphs, Temporal Graphs Privacy, Predictive Text, Confidence Gating, Valence, Affect Models, Spectral Graphs, Human-Computer Interaction

Sections, Subsections and Contents

1. Introduction
 - 1.1 Contributions
2. Related Work to IntentSpider
 - 2.1 The Sequential Processing Background of Current Keyboards
 - 2.2 Usage of two memory systems or Dual Timescale Systems and the process of learning
 - 2.3 Protecting System's already learned information
 - 2.4 Graph Diffusion System and Usage of a Personalized PageRank
 - 2.5 Background for Using Local Computation for Practical Applications
 - 2.6 Strengthening and Suppressing Edge Connections in IntentSpider (Graph Edges)
 - 2.7 Signed Graph Connections, and measuring The System's Emotional Tone Signal (or Valence)
 - 2.8 The Reading of Affect and Emotional Tone in Text Communication Between Humans
 - 2.9 Mapping and Placing Graph Connections Into a Smaller Space (Related to Spectral Graph Theory)
 - 2.10 The Novelty and Claims of IntentSpider Webnet
3. A Novel, Personal Intent Graph with Function Typed Edges
 - 3.1 Where and How This System Runs
 - 3.2 Nodes, The impact on function based on the specific thread (or silk) type
 - 3.3 Edge Role Assignment Process and Theories
 - 3.4 An Overview of the Architecture in Figure 1
4. The Process of Diffusion
 - 4.1 Heat Kernel Formulation
 - a.1 Creating Seeds from the Node Dispersion Algorithm and Using Shannon Entropy in the Information Theory to Rank Previously Typed Words by the Human
 - 4.2 Personalized PageRank Formulation
 - 4.3 The Local Push Process for Running the Diffusion Inside a Single Device
 - 4.4 Ranking the Next Words and the Usage of the Entropy Signal
 - 4.5 Geometric Paths, and a Real Time Two Dimensional Embedding of the Intent Graph
 - e.1 The Embedding as a Two Dimensional Web
 - e.2 Transforming a Sequence of Words into Paths
 - e.3 Theories related to Heads and Plateaus of the Curve
 - e.4 Two Paths in One Plane, the Shared Prefixes, and the Gap Between Them
 - e.5 The Reconvergence Process and Its Relationship With the Fan Dispersion Values
 - e.6 The Median Line, and a Question That Is Open and Available
 - e.7 The Reverse Process, and a Question That Is Open and Available
 - e.8 The Relationship With the Already Existing Entropy or Confidence Signal
 - e.9 A Worked Example With Illustrative Values
 - e.10 Background of the Subject of Study

- e.11 The Reason This Section Remains in a Careful State
- 5. The Problem Related to the Blindness to the Valency
 - 5.1 An Example With Two Selection Events
 - 5.2 Why the Existing Exhaustion Penalty Does Not Correct This Problem
- 6. Proposed Extension: Valence Gated Signed Diffusion
 - 6.1 Design Goal
 - 6.2 Scoring Valence On Device Through Typing interval patterns
 - 6.3 Signed Edges
 - 6.4 A Third, Suppressive Set, and How It Now Relates to Transmission Fidelity
 - 6.5 Signed Diffusion Read Out
 - 6.6 Algorithm
 - 6.7 The Local Push Mechanism State
 - 6.8 Task Log Based on History of Word Acceptance to the System Using Coefficients in Input Contexts
 - h.1 Introduction
 - 6.9 History Based Task Scheduling Using Coefficients From Previous Contexts
 - i.2 Residue Value
 - i.3 The Support Value From Past Selection Events
 - i.4 The Two End Values of the Interpolation Process
 - i.5 Selecting Rules in between a Fresh Word and a Residue Word
 - i.6 Note About Already Established Research Knowledge
 - 6.10 Details Related to Typing Rate Diff Values, Shared Cluster Reinforcement Process, Growing Radius Value, and Sudden Termination Process
 - j.1 The Typing Rate Difference Signal
 - j.2 Compounding Cluster Reinforcement Process Under Repeated and Prolonged activity
 - j.3 Growing Engagement Radius Under a Sustained Typing Rate Difference
 - j.4 The Sudden Negative Selection Responses
 - j.5 The Timeout Process of the Grace Period and the Settling Event (the function of returning to normal state).
 - j.6 Discovered Per User Typing Sub State Clustering Process"
 - j.7 The Process of Combining Several Typing Vector Values or Signals Into One Value
 - j.8 Scaling Process of the Communication Channel
- 7. Necessity Gating Algorithm and Suggesting Functions
 - 7.1 Using The SCAMPER Method to Generate This Process and Obtain Results
 - 7.2 The Proposed Process
 - 7.3 Relationship With the History Based Task Scheduling Process and the Complete System
 - Updated Algorithm 1 With Added Theories
- 8. Privacy Related Processes and Disadvantages
 - 8.1 Processing the System Data Inside a Single Device
 - 8.2 The Differential Privacy Process and the Research Questions
 - 8.3 Identification Through the Typing Interval Pattern Values
 - 8.4 Protection and Safe Storage of IntentSpider Graph Data

- 8.5 About Typing Rate Values and Typing Sub States
 - 9. The Processing and Storage Amounts Required Inside a Single Device
 - 9.1 Storage Requirements
 - 9.2 The Processing Time for Each Keyboard Input (character addition, backspace, delete, other actions, formatting, etc.)
 - 9.3 Comparing Against Already Existing Processes
 - 9.4 The Processing Amount of the Periodic Graph Embedding Recalculation
 - 10. Experiments and Validation
 - 10.1 Evaluation Questions and Experimental Structure
 - 10.2 Data, Splits, Metrics, and Statistical Rules
 - 10.3 Implementation Fidelity Audit
 - 10.4 Artifact Integrity and Reproducibility Boundary
 - 11. Conclusion
 - References
-

1. Introduction

Software keyboards such as modern Google Gboard, Microsoft Swiftkey, Apple QuickType, etc., predict the next word the user types from a local and sequential context by matching two, three or a "y" number of previous word units against a SLM, or a Statistical Language Model, and the most likely next words are suggested to the user.[14, 17]. If that array of words, in that order, was input to the system earlier. If the input engine did not register that particular combination or permutation of words earlier, its suggestions lose accuracy and predictive ability, since it relies on sequential processing, the system diverts to a separate, default dictionary system or an alternative method that offers generic predictions instead. It also identifies patterns stored similarly, regardless of how long it has been part of a human's stable vocabulary, compared to how recently and transiently it appeared. And it has no native process for differentiating a 'next word' that was later confirmed by the user's choice or behavior from the next word that was later rejected.

Production systems address this problem in Google's Gboard, for example, trains the underlying next word model using Federated Learning capabilities, so that information in raw forms will never be transmitted but processed internally, and only combined and averaged model updates are shared to the server. [10]. However, federated learning by itself does not resolve the empty start and context-dependent memory or Situational memory problems described there. A federated language model uses local storage for sequential memory or its sequential context, And also, because it is trained on millions or billions of devices, it is, as obvious, not personalized for each individual device or the terminal node than an engine like IntentSpider, which operates entirely individually and builds only that human's own webnet of word connections, without any involvement of external servers.

IntentSpider is an on device, personalized alternative to those existing systems. It is a system that represents an individual's words and short, dynamic sentences as nodes of an intent graph, which is private and is not shared with any external party, and each edge is scored by the accuracy, meaning that how reliably it leads to the word the user actually ends up choosing. This score is a number that can rise and fall, rather than being fixed by how long the edge has existed so far or how often it has been used. And it is based on 'dispersion'. Then, IntentSpider sends this seed to the outward through only the edges that have earned a transmission reliable type status, using a personalized, PageRank type graph diffusion [16, 13, 1]. For that reason, first time word combinations or permutations can still generate a more accurate next word if it falls near an already well-connected, reliable region of the user's personal graph (motivated by the analogy of the vibration signal reaching the spider through most reliable threads) This component replaced an IntentSpider legacy version that instead handled two separate connectors or connector types which were operating with a difference in timescales. [19] [11, 21, 2, 22]

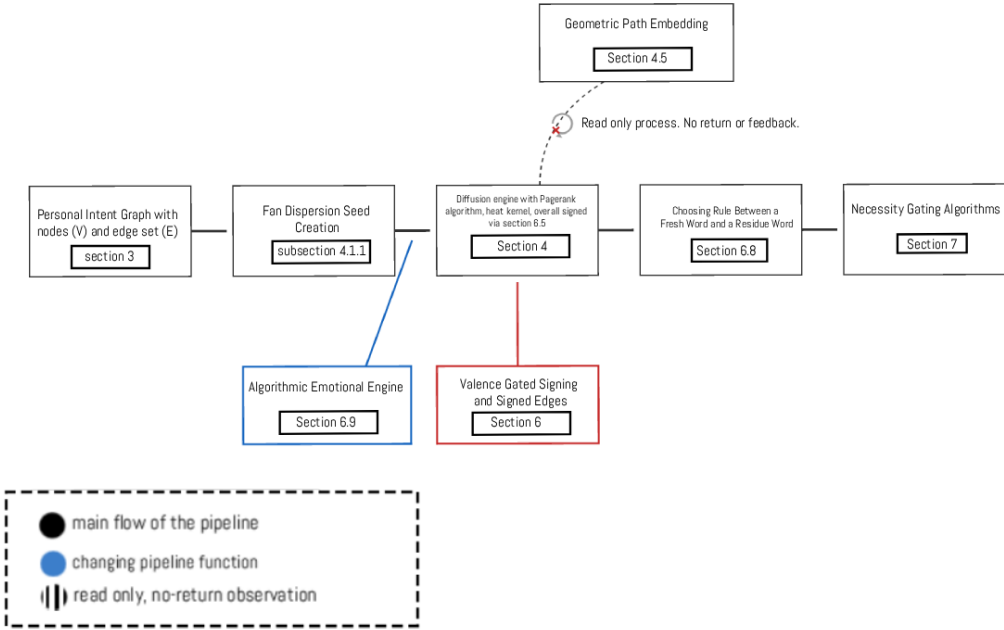


Figure 1. Overview of the IntentSpider processing architecture, showing the relationship between personalization, diffusion, valence, necessity gating, and the proposed downstream processes.

1.1 Contributions

1. It gives an accurate description of a diffusion architecture with assigned roles to them, In this architecture, edge's role in sending signals is assigned by the reliability of that edge so far, rather than using timers or clock function., And also, The novelty of the IntentSpider is compared with many other research and related work including the two timescale and elastic coefficient consolidation work (or EWC), which proposed in IntentSpider legacy version.
2. IntentSpider solves a structural problem in the IntentSpider legacy base architecture. This problem can be named as the blindness to the valency. And also, in this problem, neither the raw coefficient value update nor the reliability value update can separate a next word selection that confirms what the user meant from the next word that makes the text exchange with the other party final.
3. IntentSpider also gives a separate and independent process for filtering, which is "necessity gating" process for deciding whether a next word should be displayed or chosen. And also, this decision is separate of next word that the processes of Section 3 and Section 4 would rank first. And also, IntentSpider generates this necessity gating mechanism through a creative use of the SCAMPER method (Substitute, Combine, Adapt, Modify, Put as a one, Eliminate and Reverse, not Rearrange.), rather than from a new failure mode found in the system.
4. Our system shows a testable system with pass and fail return outputs, metrics and invented benchmarks (refer to Section 10), also, this process exists because of the reason that main claims of paper, that architecture of Section 3 through Section 6 outperforms a plain word prediction baseline (or

n-gram methods), and also outperforms relevant systems that use two memory speeds to learn (to avoid forgetting), in both cases where lacking previous history to learn and mixed valence conflict cases., and that necessity gate reduces irrelevant suggestions without slowing the entire IntentSpider system or crashing the UI, etc. This experiment will be discussed even further in the paper.

5. There is a second system that operates separately like as the necessity gating architecture. Named as the 'affective axis', this can also be named as "hexarousal" alongside the valence signal.

In addition to that, IntentSpider has six more system processes (refer Section 6.9). These built entirely from an motivation by an attacking and struggle analogy of the Orb Spider, Wandering Spider, and the Rolling Isopod prey. This includes six systems. The first is reinforcement that accumulates in across several next words in live at the same time. Another is a diffusion radius value (motivated by the vibration ripple pattern) that makes the hexarousal move to higher levels when the diffusion radius value raises high. Another is rapid suppression (within a short time). Another is an event that executes the command right after the shock eventlister triggers. This event is separate from ordinary suppression. Another is a stability rule which allows the excited state a rerouting time before it turns to the ordinary state. The other step is a unique set of values, and it finds the x number of distinct excited states that human's current typing shows. And the last is a combination of behavioral features. Also, this is the most complex and the system which contains most parameters as well.

6. Also, this gives the same graph a 2D spectral embedding (refer to section 4.5). This means, it turns each typed sentence into a path through a plane, rather than by a sequence of abstract vectors.

In the IntentSpider Webnet, we created two testable points (or hypotheses) from this embedding. First, a node's connectivity breadth predicts its distance from the center of the structure. Second, the dividing sentences connect more tightly around a shared and narrow word than around a shared, broad word. Unlike other contributions, this hypothesis does not change the system's behavior. This can be considered as a debugging or an inspection tool built from outputs that IntentSpider already produce. And this is not a merged process to any other systems or processes.

2. Related Work to IntentSpider

2.1 The Sequential Processing Background of Current Keyboards

In text prediction systems, often, there are two types of techniques common in papers. The first one is classic n gram language modeling with smoothing. Katz back off model uses a shorter context whenever the longer one is missing and/or not accessible. [14] Kneser Ney smoothing [17] Kneser-Ney method directs the probability to contexts that are more useful, not the ones which are frequent in use. They are foundational to these systems because they have a small memory and cost. Andrew Hard, affiliated with Google Gboard, trained a recurrent next word model for a mobile keyboard using the Federated Averaging algorithm. In that system, the raw input stays within the device indefinitely. [10] It only sends language model updates, combined with many other devices, that are transmitted to the server. This work partially motivated us to build the IntentSpider's privacy capabilities into a single, per-device basis structure that one device could compute entirely on its own, with no topological networking, such as without any involvement of a network. These are capabilities that Andrew Hard's Federated Averaging algorithm did not possess. Their model is limited by the sequential context limitation of the neural language model it trains. IntentSpider, conversely, is fundamentally different from that architecture as previously mentioned.

2.2 Usage of two memory systems or Dual Timescale Systems and the process of learning

The theory of two interconnected memory systems running at different speeds is a old concept. The concept did not start from a ML background. In the human brain, in cognitive neuroscience field, Complementary Learning Systems or CLS theories describes that the human hippocampus is showing these patterns:

1. fast learning speed compared to the neocortex

2. Sparse encoding or sparse neuronal representation, which means that neurons are activated in a nonlinear, way. (e.g.: playing a game) It activates the occipital lobe, Parietal Lobe, PFC and frontal lobe, in an unpredictable, sparse pattern. [11]

3. And changes by the incident type in a multidimensional way, as mentioned, it is not a predictable pattern. And it depends on billions of parameter that are dictated by the laws of physics. So it is individual and incident specific.

Conversely, the human neocortex operates in a learning memory method that is slow, statistical, and very distributed instead of being centric. In that way, a proper way to arrive at a conclusion is an electronic system that only learns fast are more likely to forget fast, and a system that only relies on slow learning are much prone to slow learning and in rare cases, memory overloads or SIGTERM errors. (C++ runtime error)

Professor Geoffrey Hinton and Dr. David Plaut [11] described "Connectionist networks" (referred as ANN or Artificial Neural Networks) where every connection is in the "true" state, and carries two coefficients. Each network in the Connectionist network consists of a slowly changing coefficient that stores long term knowledge, and a connected fast changing coefficient storing temporary knowledge and decaying back to zero on its own. Scientists showed that practicing with a small amount of associations is sufficient for the use. This lets the fast coefficients cancel out the interference the slow coefficients had built up.

Also, (in Section 3.2), it replaces the split with a single function based linking set (or an edges set). And also, this research area is under active development currently. The specialty of this field is that it is being under sustained research and development since historically. Which matters for the novelty claim. Three recent systems show this same fast and slow pattern in different types of forms. This includes the DualNet, from Dr. Quang Pham at el's research, [21], Dual-Net splits the tasks into two separate learning processors, This includes a fast process that trains itself (without labeled data) and a slow process that preserves the learned data and information using Elastic coefficient Consolidation. Complementary Learning Systems based Experience Replay (or CLS ER), manages two similar clones or copies of each network and then each copy updated at a variable number of speeds. Then CLS ER uses how much amount the two network copies do not differ on a example run as a signal for which specific previous examples to replay. This implies that the system has a priority for the greater differences. [2], ITDMS (Refer Information Theoretic Dual Memory System), this system keeps two separate memory buffers separately which includes a fast memory buffer for new sample runs and a slow memory buffer for samples that has the long term value for runs, This choosing process is based on the information theory(ITMO methods as a subset of the Information theory) (refer [22])

These above 3 research papers were published between 2021 and 2025. Which were built on the CLS theory foundations. And all three use some version of the same fast and slow split an earlier version of this paper also used. Which means that, a novelty claim based only on that specific two channel structure methods will be weaker, compared to the already existing research.

2.3 Protecting System's already learned information

Elastic coefficient Consolidation or EWC supplied the mechanism an earlier version of this paper used for protecting an edge once it moved into a stable channel. (refer [15]) For each parameter, this EWC estimates that how much changing it would damage the performance levels on tasks the system already learned so far. This is an estimate that uses the theory "diagonal fisher information approximation". And this method makes changes to this in the future difficult and more complex. This specific resistance only increases. Once a parameter, or in the legacy IntentSpider version, an edge, is considered as "important". And therefore, EWC has no ability to reverse or change that previous state of importance. Refer to section 3.2, it states that, why this one way property is a disadvantage for the system. It should not be considered as a feature for a real-time, live and a personal graph (or per user graph). In such a graph, These types of connections should be able to remove their 'important' state with the same amount of frequency of instructions or the effort similar to how they obtained it. In this paper, EWC is only used in this paper as the source for baseline B3, Which is one of the comparison systems used in later sections (refer to Section 10.2). EWC also matters because Section 3.2's own "freely reversible" property is defined by contrast with it. In the IntentSpider system, this does not use EWC as a system mechanism.

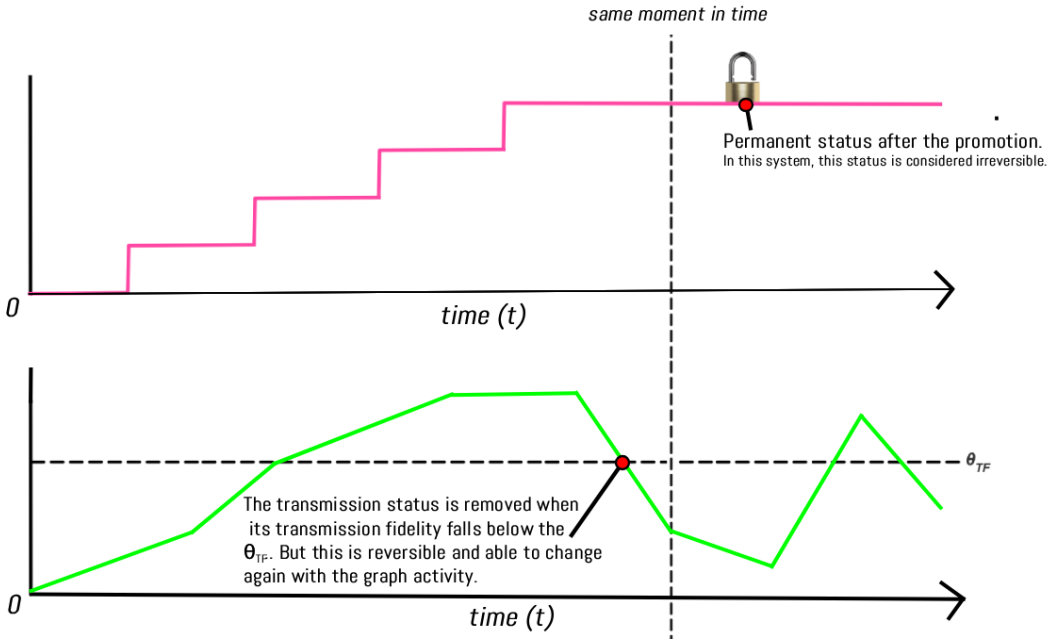


Figure 2. Illustration of reversible transmission-fidelity state changes and the settling of an active transmission state after the relevant activity terminates.

2.4 Graph Diffusion System and Usage of a Personalized PageRank

Personalized PageRank builds on the original PageRank algorithm, which was developed by Larry Page and Sergey Brin [20], (more specifically its random walk model.) (refer [20]).In here, instead of restarting the process similarly across the entire graph, it restarts from one specific initial node, or a small set of initial nodes [13].Johannes Gasteiger et al [16] showed that this method, used on its own without the neural network's prediction step, improves how well nodes get sorted into groups when only some of them are labeled (which is known as the semi supervised node categorization in fundamental ML), also known as PPNP or APPNP. This gives one example of using diffusion by itself, rather than a trained classifier, as the last step in Section 4. This gives one example of a case where diffusion works without involvement with other system processes. It functions more than a trained categorizing engine as the last step described in the Section 4. In addition to that, there is a similar option for it which is called as the heat kernel formulation (refer section no. 4). Professor Fan Chung [4] showed that a if this process runs for the right amount of time, it turns into a variation of the PageRank algorithm previously discussed. Because of these reasons, the two constructions used previously in section 4 (refer Section 4.2 and section 4.3) are the same core idea. In that case, only difference between them is how long this diffusion process has been allowed to run in the system.

2.5 Background for Using Local Computation for Practical Applications

IntentSpider's use in PageRank variation random walk model, and computing an exact user based PageRank vector means solving a system of linear equations

across the entire graph here. This requires a massive amount of processing power (the actual runtime time would depend on the specific device) if done repeatedly. This is not a capability a mobile phone can practically repeat on every single keypress input Reid Andersen et al. solved a version of this problem previously in 2006 (refer [1]). Their local push algorithm computes an approximate personalized PageRank vector instead, by pushing mass outward from an initial node. Here, In this process, 'mass' is only added to nodes which still carry more than a small, fixed amount of unused value. And also, the entire process finalizes and ends in time which is proportional to $\frac{1}{\alpha\epsilon}$ and this time is not dependent on the number of nodes on the graph. This is the reason that in section 9, we claim that this system is possible and capable of running inside a mobile device like an iPhone. Without that, the diffusion step would need to process every word, word by word a user has typed so far, and also, for every single keypress. Making it unrealistic.

2.6 Strengthening and Suppressing Edge Connections in IntentSpider (Graph Edges)

Ant Colony Optimization, or ACO, takes a massive space in terms of initial contributions to the IntentSpider development and research, which means that the edge strengthening and edge decay dynamics used for the raw coefficient system in Section 3.2, this methodology came directly from the Ant Colony Optimization (refer [5]). In this method, pheromone trails on a graph get reinforced by traffic (the analogy of ants moving through a field of clay), and these trails of the ants' foot fade according to the time passed so far when they are not used. And also, in addition to that point, After a pattern completes, the exhaustion penalty value temporarily blocks it from activating again. The after that, it remains restricted until that suppression period ends. This penalty system is derived directly from the research related to the Tabu mathematical search optimization stacking algorithm or commonly, "TS" (refer citation [9]), Which is a process that also blocks a the system from retaking the already and recently used value or the solution in general terms, for a fixed constant amount of time. In IntentSpider, this blocking system exists to stop the engine from choosing the exact same next word being chosen again and again. (refer to section 3.2, 5.2 and section 7)

2.7 Signed Graph Connections, and measuring The System's Emotional Tone Signal (or Valence)

In Section 6, it needs a programmable object for a type of graph. In this specific graph, some edges should actively push back against a spreading signal. And also, this push back is different from the general type of fail. Which means that it is different from the failure to reinforce that signal. In the fundamental graph theory in math, this is not novel. Professor Dorwin Cartwright and Professor Frank Harary (refer [3]) turned Fritz Heider's theory, which is the way how society and its components, which means people manage to keep their attitudes and preferences in balance and consistent with the one of the great theories that used to build a structural balance theory. And also, they built this theory specifically for signed graphs. In addition to that, Jerome Kunegis and others (Refer [18]) proved a valuable information. The standard spectral graph theory methods is working with graphs with edges with the value less than 0.0 (zero).

In addition to that, this method includes the the graph connection matrix (also known as the laplacian), spectral clustering, and diffusion kernels. And also, in addition to that point, after doing this, there are several implications that happen afterwards. In addition to that, the IntentSpider legacy version required our previous method to produce the valence signal using a lightcoefficient and an already available source only. Which means that, typing speed and correction behavior can be given as examples for this lightcoefficient signals. These signals can be derived from the device directly instead of using large computational needs for data. And also, this process needs to happen inside the device. That previous version used VADER (referenced in research as "Valence Aware Dictionary and Sentiment Reasoner"), which was a sentiment model based. (refer Dr. Clayton and Dr. Gilbert's VADER model [12]). In addition to that, VADER is not a neural network based model. It is a rule based model and the model VADER was built by the scientists to run it without data servers. Unlike the IntentSpider legacy version, in the upcoming section 6.2 now replaces VADER with a separate. This new signal is based on the specific pattern of the typing speed of the human, and also, it does not require a list of words which are created early. This new signal is motivated by the spider neuroethology study, which researches how a spider's nervous system produces responses (refer [26]). It is not motivated by VADER's research on reading emotional meaning from words that are being typed or input. And since the VADER research paper was a main source which was used to construct the theory, IntentSpider's this version also cites that specific paper. And also, VADER research is the reason why the section 12 can name sarcasm and indirect refusal handling as a specific limitation. This is a limitation the word list approach would have carried. In that kind of a case, VADER was built and tested for texts and lines found in social media posts, and text posts in general. (using data obtained by Twitter or now X). And it was not built or validated for this level of precise data harvesting, meaning that, keypress level Intent Gating. Which implies that the accuracy of it in this more narrower study of IntentSpider was never tested.

2.8 The Reading of Affect and Emotional Tone in Text Communication Between Humans

Aligned with this, there are two already existing research papers that support the Affective framework which is being discussed in Section 6.9, and here, these two citations also support the warning of the calibration discussed later in this current section. After that, the first step is representing the Affect as a very small space included with independent axes, meaning that, axes that do not correlate with each other rather than as one single number. Professor James Russell identified and proposed representing (refer [28]) the affect in this case, two independent dimensions, valence (pleasant axis to unpleasant axis) and arousal (activated axis to deactivated axis), arranged into a circular model, which means that circular model connecting the two dimensions. This specific research belongs to 1980. (refer section 6.9'1.) In addition to that, Professor Joseph Walther introduced the "Hyperpersonal Model" (refer [29]) which is an already exciting theory which names how online text communication is different to which specific parameters when compared to real life human chat. And this research establishes something very specific. Which means that the emotional nuances and the small obvious points in the communication is depending on

the specific medium of the communication used. Which means, that difference changes differ with whether it is email, note taking, web messaging, or a Google search. Section 6 (specifically Section 6.9.8) is explaining this with factual basis. There is a typing rhythm based arousal or valence signal, hence, it is calibrated on a one medium, such as Instant text messaging (Skype, Signal). Due to that reason, this custom settings should not be equal to or substituted directly without editing to another medium's settings, such as Email or Twitter. Hence, this custom setting's value is left unspecified below. Due to that reason, it is not assigned a value it has not earned. But not any of these citations support the other six mechanisms in the section 6.9. Six of them are, compounding reinforcement, growing radius, shock retreat, settling, sub state clustering, and the nonlinear interaction coefficient. These six mechanisms can be considered as invented mechanisms.

2.9 Mapping and Placing Graph Connections Into a Smaller Space (Related to Spectral Graph Theory)

In the Section 4 of the IntentSpider paper, the Section 4.5 is an already existing idea. Which demonstrates that from the eigenvectors of its Laplacian, a graph's structure can be recovered in a low dimensional form. In addition to that, Professor Miroslav Fiedler [30] researched and found that The eigenvector tied to a graph laplacian's second smallest value of eigenvalue gives a one number as a numeric for how much is the interconnectedness of the entire graph. Which means that, a higher value implies that the graph is more challenging to disconnect into different sub parts. Which is now known by researchers as "the Fiedler vector". Because of this one result, this specific technique makes this possible to use Laplacian eigenvectors for separating a graph or connecting into it. Professor Mikhail Belkin and Professor Partha Niyogi [31] built the specific technique Section 4.5.1 uses directly. They represent each node by its coordinates in the eigenspace of the top few non trivial Laplacian eigenvectors. This produces a low dimensional embedding that keeps local graph structure in place. It is related to, but distinct from, PCA or classical multidimensional scaling. Section 4.5 applies this established technique to a live, per user, keypress evolving personal intent graph. From this, it pulls two testable geometric predictions about connectivity and the reconvergence. As said, from those already established researches, non of those researches creates or implies those specific hypotheses used in the IntentSpider system.

2.10 The Novelty and Claims of IntentSpider Webnet

After reading from section 2,2 to 2.9, One of the conclusions that we will be able to take is that, the text prediction and more specifically, the on device text prediction is a established area of research in human computer interaction. In addition to that, the theory of Two timescale memory architecture is an already existing area of research study with about minimum of three research architectures published from 2020 onward until 2025. Those architectures are also using the CLS (or Complimentary learning systems theory, methods) architecture used in the base of the IntentSpider as well. In addition to that, protecting learned parameters, diffusion based classification, local computation inside the device, reinforcement and dynamics of inhibition , signed graph methods, keypress dynamics biometrics, and spectral graph embedding are also previously

researched concepts into a some extent. And also according to several biological findings related to Orb weavers and the research related to it support these ideas as well. Which means orb weaver silk comes in separate types, each type of silk is for a one specific function. Tiger wandering spider's (scientific name: *Cupiennius salei*) frequency dependent approaches, and its withdrawl response and Professor Russell's circumplex model related to the valence and arousal.

We have not found a system that types graph edges by an ongoing, freely reversible measure of whether they actually predict what a user goes on to select. This is different from typing edges by elapsed time, usage frequency, or a one way consolidation clock. And also, we have not found a system that reads that typed graph out through diffusion, seeded by a combination of several recent words coefficient based by how scattered their connections are, rather than by the single most recent word. Also, we have not found one that adds a signed, valence gated extension whose valence signal comes from behavioral timing rather than word content. That is some of the claims that IntentSpider Web-net paper makes. And the research behind the claim above was not repeated specifically against the affective-computing and text-based computer-mediated-communication (CMC) research of Section 2.8, for compounding reinforcement, growing engagement radius, shock-retreat, grace-period settling, or emergent sub-state clustering. In section 11.2, it shows that this component's contribution to the paper's overall novelty case separately, and as clearly weaker than the four components the claim above actually covers. Nor does the claim extend to Section 4.5's geometric paths, for a different reason. This is not a claim of unverified novelty. It is a claim of a different kind entirely. The claim above is about a predictive architecture, what gets suggested and how. Section 4.5 predicts nothing. It observes and measures what the existing architecture already produces. Whether representing a live, per user diffusion process as a two dimensional keypress path has been done before, in exactly this form, was not the subject of a dedicated search for this revision. Section 11.2 treats this component on its own terms, as a candidate way to look inside the system, not as a candidate improvement to prediction quality.

3. A Novel, Personal Intent Graph with Function Typed Edges

3.1 Where and How This System Runs

IntentSpider system is operating overall inside a single device. And for a single user. With IntentSpider, the server side components, external servers to transceivers are absent. And also, there is no combining of data across multiple different users. This is not only just a preference but a self imposed constraint in design. This structure of IntentSpider makes the privacy arguments and concern in Section 8 minimal and well regulated, even without the need of a cryptographic or differential privacy proof (examples for DP include Apple Quicktype, Apple IOS and examples for cryptographic privacy proofs include Signal messenger, Coinbase's wallet). Which implies that the graph described below is built from only that specific human's personal typing history. And it is as mentioned, not transmitted to any other separate server via any medium. Those data and/or information are not synced internally in a readable form as well also, those data are not used to update or improve or accumulate into any other larger Webnet or a network.

3.2 Nodes, The impact on function based on the specific thread (or silk) type

Orb weaver spider webs are built from structurally and also physically distinct silk types. And each of those types of thread is for a distinct and a specific purpose. Radial threads and frame threads are created using a thick and a dry silk. Those thread types support the geometric structure of the spider web. And also these threads transfer and transmit the vibration signals again, back to the spider that produced them. But this spider web's spiral, which is used to catch the prey, has a very low thickness and a high flexibility, and also the ability to hold (or the sticky described in Dr Foelix's paper as "Sticky"). These are the type of spider silk that is also strengthened by the chemical gland of the orb weaver spider, which produces the silk. Which means that the purpose of this type of thread is to stably hold and keep specific data objects. Which means that, the specific thread's purpose or the role is not decided by the time it has existed, but instead, it is by the specific physical properties of that thread. This motivates replacing a time based channel split with a function based one. An edge's role in the graph is determined by whether it reliably carries signal to a useful place, not by how much accumulated support value it has built up over time. Let V be the set of tokens, meaning words or short word groups, that this specific user has typed. This set grows as new tokens appear. Rather than two coefficient functions across two separate edge sets, IntentSpider now keeps a single edge set " E " on " V ". Every edge, referred to as u and v , in this set carries two values, and each of those values independently updates itself. Raw coefficient. In each of those selection events, meaning the user accepts a suggested continuation v following context u , the raw coefficient is reinforced, (or strengthened). In other cases, it starts to decay in exponential value, according to the time since it was last used. This works in the same way as the pheromone trails of the ants inside the Ant Colony Optimization algorithm (or ACO algorithm) (refer to [5]).

$$w(u, v, t + \Delta t) = w(u, v, t) \cdot e^{-\Delta t / \tau_v} + \eta \cdot \mathbb{I}[\text{selection event at } t + \Delta t] \quad (1)$$

In this context ' τ_v ' represents a short sized decaying constant (referred as Tau sub V constant) which is programmed to be active between several hours and

several days. This specific speed of the decaying correlates and decides how much fast the strength of a word connection lowers or decays when a user does not choose it for a specific period of time. And also, this method makes it sustainable by programming the tokens so the per user graph remains dynamic by force deleting earlier, old associations to allocate space for new patterns. This specific symbol ' η ' refers to the relevant learning rate used by this process. This learning rate denotes and correlates with the the amount of the coefficient of a specific connection increases each single time when a human selects a word. Which means, in this method, this successfully controls the speed with which the engine adapts to new typing patterns and learns the human person's personal preferences.

In addition to that, this above equation (1) updates one single edge per each selection event, Which means, based on the specific moment when a human accepts a suggestion, below in Section 6.9.2, this paper introduces an additional word group strengthening term. This term layers on top of the existing update, rather than replacing it, and it only applies when several candidates are continuously chosen by user and also while live at the same time. This specific new reinforcement component uses its own initial reinforcement value. And in here, there is a specific separate symbol for this value as well in order to avoid the confusion points with η in other calculations.

To manage the overload ("exhaustion" in this equation) our system uses Tabu Search algorithm which was developed by Prof. Glover and which is one of the most core and important features as it was in the legacy IntentSpider version. This specific method with Tabu search prevents the system from suggesting the same word too many times. And as the section 2.6 demonstrates, After a specific selection event occurs by a human, the system blocks that edge for a variable amount of time. And this specific blocking process (in this case, this blocking process and reinforcement is separate and not correlating each other.) is a separate process.

$$\text{exhaustion}(u, v, t) = \exp \left(-\frac{t - t_{\text{last}}(u, v)}{\tau_x} \right) \quad (14)$$

In this Specific case, ' $t_{\text{last}}(u, v)$ ' represents the time of the most recent selection event on edge (u,v), and τ_x represents a short suppression period, which Section 2.6 explains in more detail (refer [9]). This τ_x symbol is kept as a separate notation than the " τ_v " notation which correlates and decides in which kind of speed the raw coefficient decays in that Equation (1), and also separate from the recent constant τ_0 , (which is the constant that is introduced in the section 6.8), where it paces the arbitration support decay instead. $\text{exhaustion}(u, v, t)$ is a separate accounting signal by itself. (As a note, this specific signal remains closer to the 1 that specific moment after the selection. and as the time spends, that specific value starts to decay down to 0). It is not subtracted from $w(u, v)$ in Equation (1). And the Section 7 uses this signal downstream. In section 7, there is a signal in order to decide whether a candidate should be shown but it is not used in order to change any already stored edge coefficient.

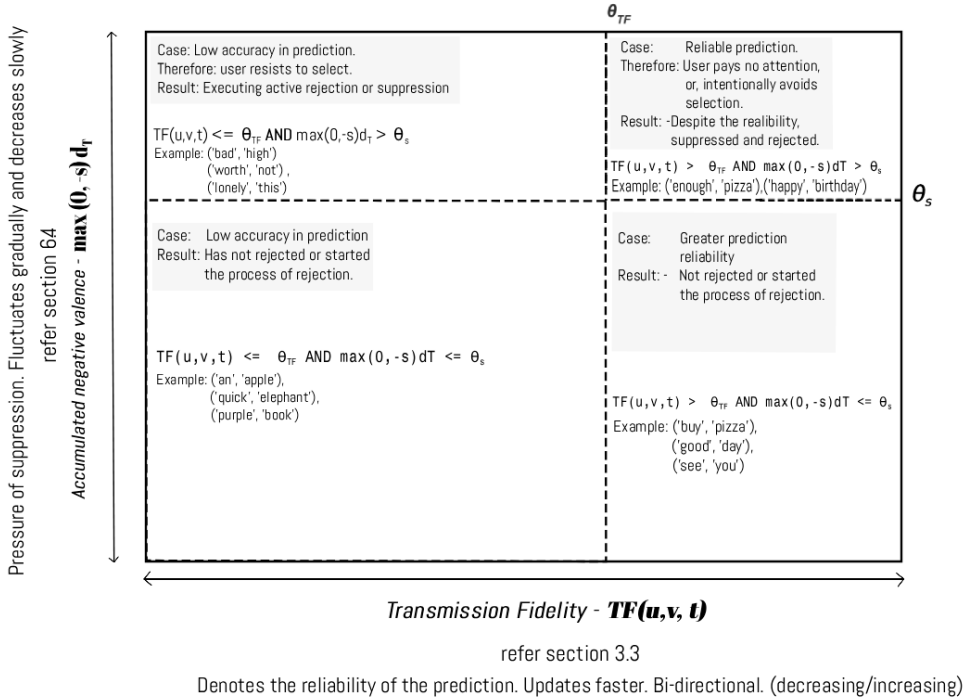


Figure 3. Illustration of transmission fidelity as a measure of prediction reliability, including the distinction between low- and high-reliability edge behavior.

This section introduces and defines transmission fidelity. in this theory, every edge object includes and has assigned to a transmission fidelity value, which is written here short as "TF", and this specific value is built with u , v and t values and the TF value if always between 0 and 1. (As a note, this specific TF value only updates itself at that moment the connection sends a specific amount of diffusion mass to a next word the human selected after that event.)

$$TF(u, v, t + \Delta t) = TF(u, v, t) + \eta_{TF} \cdot (\text{outcome}(u, v, t) - TF(u, v, t)) \quad (16)$$

3.3 Edge Role Assignment Process and Theories

$$(u, v) \text{ is } \begin{cases} \text{transmission type} & \text{if } TF(u, v, t) > \theta_{TF} \\ \text{capture type} & \text{otherwise} \end{cases}$$

Equation explanation - In here, the transmission type edge is included in the diffusion matrix $W_{\text{transmission}}$ and enables the diffusion process. The capture type edge is excluded from the matrix and is added and placed only as a direct and a non forwarding association.

The EWC's importance value which is referred as 'F' takes in u value and v value combined and afterwards, this value is only increasing the equation 2 (Eq. (2)) which was in the IntentSpider legacy version. This specific process resulted in the edge that was complex, limited and reduced the ability of the capacity

of the edges to change. But the TF val, (transmission fidelity value) and then processing value "v", "u" and "t" as a single array of data (by combining) can move in either of those directions as the human's typing patterns change. And also, an edge that fails to provide the value to the system results in a reduction from its transmission status. Which means that non of those edges are permanent in this system. After that, The legacy IntentSpider version was not capable of unlearning the already learned but falsely stable and also outdated associations and information due to a specific lack of a process for automatic and more natural cleaning. This is also discussed in section 2.3 and equation no. (2) and no. (3). To recorrect this issue, equation (16) and equation (17) invents a new process for unlearning.

This section is about the reliability of above processes. A new edge with a consistent log success history is converted into a type of transmission after a brief of time. In that same way, But also, an edge that is being used frequently but has very low (or zero) confirmed selections cannot be converted into a transmission type regardless of the specific timeframe or the age of the edge. In addition to that, the legacy IntentSpider version tracked the recent and the reliability as a combined object on a single promotion clock, This IntentSpider version separates them into separate quantities.

This section uses η_{TF} as the main free parameter for this process. This processing system does not require a fixed window "T" since "TF" calculates an ongoing exponential moving average instead of a specific windowed count. And also, compared to the legacy version of IntentSpider (means, before the revision), this system reduces the number of free parameters by one

3.4 An Overview of the Architecture in Figure 1

In here, a new input for example, such as a pizza order, accumulates a raw coefficient and after that begins obtaining a transmission fidelity value (alt. referred as the TF Score). An edge whose transmission fidelity passes θ_{TF} becomes a "transmission type" and added to the process diffusion (refer section 4). An edge that does not pass this specific margin value is a "capture type" and then remains available only as a direct, local, immediate and non forwarding association. This architecture is different from the previous architecture with two channels. And unlike in legacy version, in this architecture, movement between these two roles is continuous and reversible in both directions as $TF(u, v, t)$ value changes.

4. The Process of Diffusion

The diffusion step decides how a newly typed word, or several of those words, produces the ranked suggestions. In this process, transmission type edges ($W_{transmission}$, refer to Section 3.3) are used only for propagation. And also, capture type edges absorb new input, but those capture type edges are intentionally excluded from the long range diffusion. Meaning a transient, situational, or unreliable pattern cannot flood the suggestions in the same process a reliably predictive one can. As a note, this paragraph, and also the heat kernel and the personalized PageRank equations below, are unchanged from the earlier two channel version of IntentSpider Legacy version.

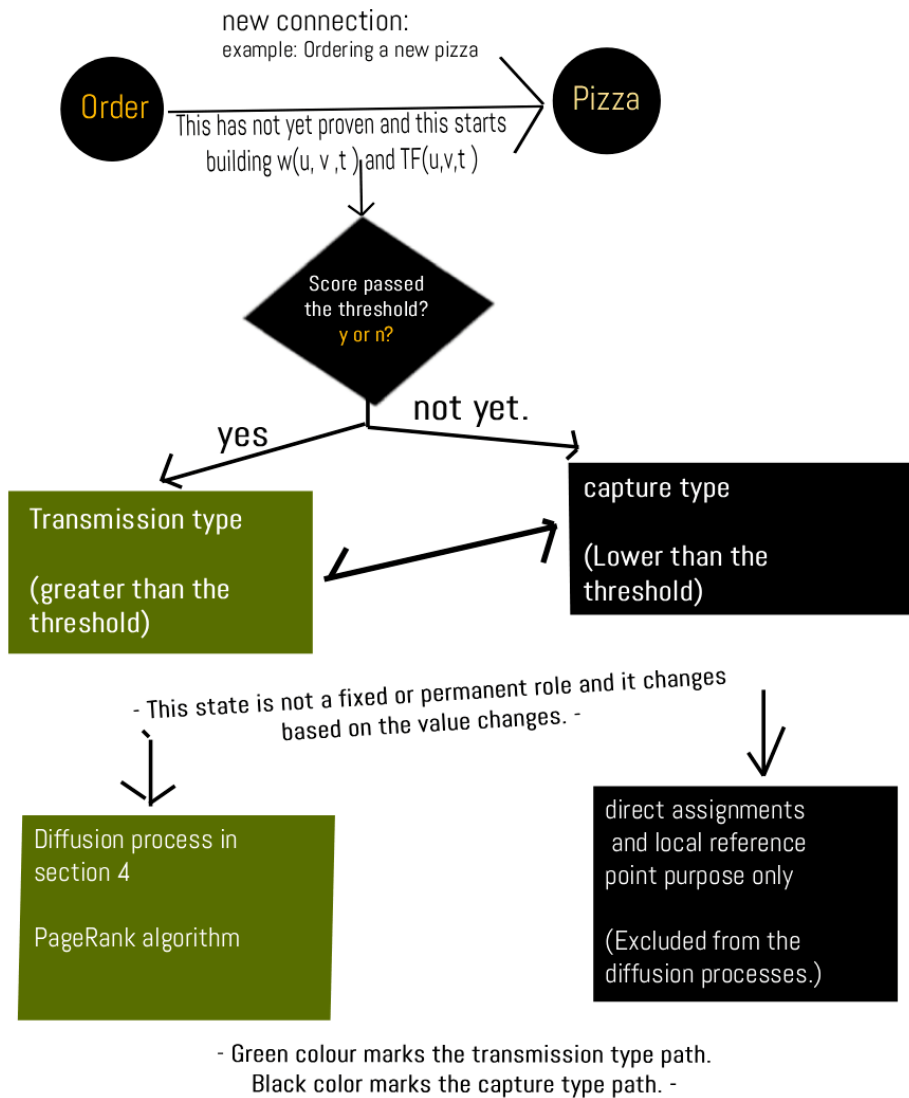


Figure 4. Figure 1 shows the single, functional and function typed edge set and the two possible states for the edge.

4.1 Heat Kernel Formulation

Let $L = D - W_{\text{transmission}}$ be the graph Laplacian of the transmission type edge set, and in this case, D is its degree matrix. The heat kernel at the diffusion time t is

$$H_t = \exp(-tL), \quad s(t) = H_t \cdot s(0) \quad (4)$$

In this scenario, $s(0)$ is a seed vector, which is constructed as described in Section 4.1.1 right below it, rather than as a 0 or 1 value indicator on the single currently active word. Which means that, as "t" value increases, the signal is shared via using the transmission type edges. And also, nodes which are strongly connected and nearby receive relatively more signal than the ones in the further distance. Professor Chung (refer [4]) noted the precise correspondence between this continuous time process and PageRank algorithm. And this specific correspondence is the reason which licenses treating the two formulations in this section as can be substitutable for one and the another.

4.1.1 Creating Seeds from the Node Dispersion Algorithm and Using Shannon Entropy in the Information Theory to Rank Previously Typed Words by the Human

A seed vector with a single 1 at the just typed word is considering every word as a equal and a similar origin or a starting point for the diffusion processes. Regardless of the specific state of the connection or the connectivity of that specific word, this process executes. Meaning that, this method does not consider and it actively does not include whether the usefulness of the specific signal that it carry. This also does not include whether the word connects to many other words that the combined and collective signal is not clear. In addition to that, in English, connecting words like 'the' or 'in,' and with other completing or filling sentences such as 'you know', 'of course', 'by the way' are soem specific examples of this problem. Which implies these objects (sentences or words can be considered in a high probability and less variance, as they can be used before or after many types of words.). Due to that reason, if one of these words is used by itself to begin this process, the signal reaches every part of the entire graph in equal amounts. And it is not moving to the human's underlying intent.

For node u , let $\pi(u, \cdot)$ be the its relative outgoing coefficient distribution, limited to the $W_{\text{transmission}}$ edges. This limit is the specific point of integration with the previous subsection no. 3.2. Meaning that, in here, the system only measures the dispersion values for the edges which are already in the state "transmission type" Hence, a relevant node's dispersion value is showing the correlating shape of its reliable connectivity. And it does not showcase the raw type connectivity(without shape) .

If we define

$$\varphi(u) = \frac{H(\pi(u, \cdot))}{\log(\deg_{\text{transmission}}(u))} \in [0, 1] \quad (18)$$

This uses the "Shannon Entropy " which is a basic theory discussed in the Information Theory. And before this, we introduced Shannon entropy $H(\cdot)$. It is also for the ranking confidence signal in the section 4.4 below. And for a second role, it is now again reused. Means that, this does not evaluate the confidence values of the output distribution but rather, it is the dispersion of the specific

input node's internal local connections profile. If a word's transmission edges spread evenly across many other words, $\varphi(u)$ equals a value which is close to 1 approx. If a word's transmission edges focus on a few amounts of next words, $\varphi(u)$ equals the value close to approx. 0.

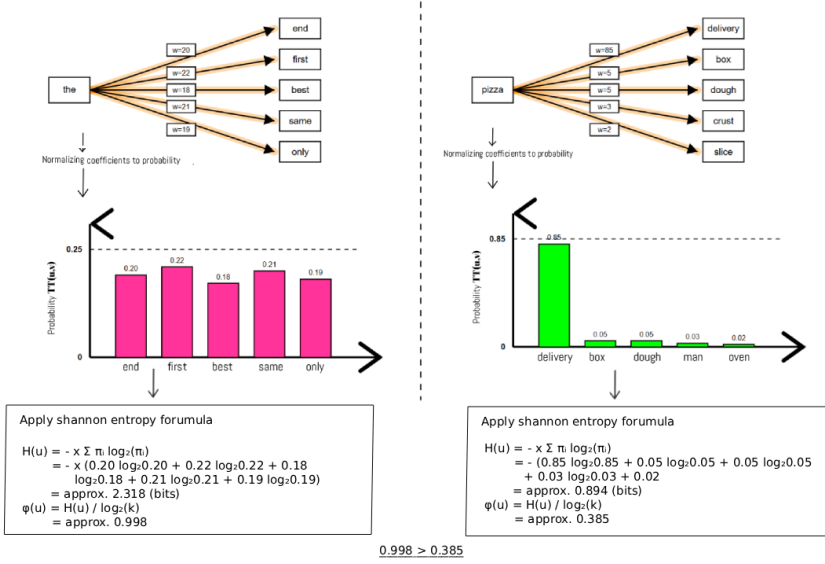


Figure 5. Illustrative comparison of high- and low-dispersion node connectivity and the corresponding Shannon-entropy values used in the seed construction process.

The seed vector for the last "k" typed words $u_t, u_{t-1}, \dots, u_{t-k+1}$ is then

$$s(0) = \text{normalize} \left(\sum_i (1 - \varphi(u_{t-i+1})) \cdot e^{-(i-1)/\tau_{seed}} \cdot e_{u_{t-i+1}} \right) \quad (19)$$

τ_{seed} does not share a symbol with any wall clock decay constant in any other place as well. Those are τ_v , τ_x , and τ_0 . And the subsection 6.8.3 is similar for ρ and γ , for that similar reason. τ_{seed} is reducing the word's specific role or the part in the seed as that word's position in the typed sequence moves further back. But it does not reduce based on elapsed real time. A word with no transmission type edges has an undefined $\varphi(u)$ value. And this word does not need a special case in Eq. (19). Which means that, this kind of a word cannot send signal. Due to that reason, it does not matter what seed coefficient the equation gives to this word. And the equation naturally gives this word no effect.

Whether the words that are directly next to each other should be combined into a single compound node before Equation (18) and Equation (19) process them is not solved by the theories and processes described above. Means, this is a question that is open and available. Meaning that a two word group such as "you know" or "I mean" or "in fact" or "for sure" or "no way" would output one dispersion value, and not two separate values. In addition to that, an extension that is not tested would use low entropy at the Bigram transition level ($u \rightarrow v$ used every time) as the method for combining u, v into one node. Means it uses the entropy tool for a third distinct purpose (wordization). This is separate from

the ranking confidence and the seed coefficienting. Also this can be considered as an unverified addition to the system. And this is not related to or derived from Equation (18) and Equation (19). This part is also listed in the section 12 as well.

4.2 Personalized PageRank Formulation

The practical, on device version replaces the matrix exponential with a fixed point equation. Let $P = D_{\text{transmission}}^{-1} \cdot W_{\text{transmission}}$ be the row normalized transition matrix of the transmission type edge set. The personalized PageRank vector p satisfies

$$p = \alpha e_0 + (1 - \alpha) P^T p \quad (5)$$

In this context, e_0 is now the fan dispersion coefficient based seed of Equation (19). And it is not a indicator vector on a single input word. This is the same change done to $s(0)$ above. And this is mentioned here again because of a specific reason. Equation (5)'s e_i and Equation (4)'s $s(0)$ are the same seed. Professor Fan Chung (refer [4]) showed that those two are the same seed at different points on a diffusion time axis. And also, both of them need to update together. If one of them is not updated, the other should not be updated either. In addition to that, $\alpha \in (0, 1)$ still controls distance this process pushes the signal before it restarts back to the seed. And this is the method also known as PPNP or APPNP (refer [16]), and, in general terms, the personalized web search (refer [13]) as well.

In subsection 6.9.3 below, The IntentSpider system considers α as a fixed constant value which is visible in above as well. In addition to that, section 6.9.3 proposes a different process. In this specific process, an effective restart parameter lowers on each keypress, as a saturating function of the timeframe and for what amount of time the a user's typing remained in a sustained state in arousal. Meaning that, this process increases or extends the signal more further when it is a sustained engagement. meaning, the walk runs more longer initially before the state restarts back to the seed state. And this uses a separate symbol (α_{base} , $\alpha_{\text{effective}}$) that does not replace the α used here. (and should not be confused) In addition to that,

4.3 The Local Push Process for Running the Diffusion Inside a Single Device

Solving this Equation 5 requires inverting an $|V| \times |V|$ system. meaning that, this is not a capability a mobile phone can practically repeat on every single keypress. This is because $|V|$ can increase into over 10,000 or even a greater amount of words for a daily computer user across several years. In addition to that, Professor Reid Andersen and others (refer [1]) give an algorithm for this specific problem. This algorithm is named as "the local push." In this method, the process sets an approximate solution $\hat{p}$ and a residual vector r instead. And after that, it repeatedly pushes mass from any node which holds a residual value greater than ϵ to its nearby neighbor values. And this process ends once none of the residual values remain over that margin value. Their central result is a runtime limiting value of

$$O\left(\frac{1}{\alpha\epsilon}\right) \quad (6)$$

And this limiting value is separate from $|V|$. It depends only on distance the signal actually needs to travel. And also, on how precisely it needs to be resolved. This is the property this architecture depends on for running the diffusion process inside a single device. Section 9 gives the quantified values for this point. The processing amount scales with the shape of the local neighborhood region area around the just typed word or the words. And it does not scale with the amount of the full typing history of that specific human.

Section 6.9.3 below adds a third point into this similar bound. And this point is a separate type than the two above. The first two are the signed coefficients question and the question about the edge set changing on each keypress. Those two points ask whether local push finalizes and ends in a fixed amount of time under those changed graph conditions. Which means, whether local push finalizes and ends once the graph it runs on holds values which are not negative values, and once the edges in the transmission type edge set are not the same on each keypress. In addition to that, the growing radius process in the sub-section 6.9.3 is different from those two. It does not change the α value into an unusable value during one finite keypress period. Instead, it deliberately reduces the α value used by the bound, as the typing of that human remains in a sustained engagement. And the processing amount increases when that value reduces. Due to that reason, this is a processing disadvantage. And it is not a question about whether local push finalizes and ends. Hence, this system keeps it as a separate point. And it does not combine it into the similar open question as the other two. (refer section 6.9.3 and the section 9.2)

4.4 Ranking the Next Words and the Usage of the Entropy Signal

The next words are ranked by the relevant resulting values in p (or $s(t)$). And they are not ranked by a condition such as the specific proximity to the input. So, whether a word was directly next to the input word is not used in here. In addition to that, there are two implications that happen afterwards from this process.

1. A word with no direct co occurrence history with the input can still be a high ranked suggestion. This state is viable when that specific word is in a interconnected region area of the transmission type graph which the relevant seed reaches. And this specific state, where the system is starting from a lower amount of data, is named in here as the cold start state. This state contributes to the motivation of this whole process.
2. The previously stated Shannon entropy of the resulting distribution, $H(p) = -\sum_v p_v \log p_v$, is a usable confidence signal. A "p" which is focused on a one intent depicts the intent values are more clear. And a p which is high in entropy depicts the intent is not clear. In that case, the system uses this value for deciding whether to display one single suggestion, or several separate next words. This is the first use of the entropy signal in the IntentSpider system. And this system now includes three separate uses of it. The second one is $\varphi(u)$ in the equation no. (18). It measures an input word's own fan out values. And it does not measure the specific shape of an output distribution.

The third one is not a verified point so far. It is in the Section 4.1.1 as a question that is open and available. All three of those are the similar mathematical object which is applied to three different distribution processes. Due to that reason, they should not be confused.

Section 4.5.8 below introduces a geometric option for the similar confidence question this specific entropy signal already answers. Meaning that, the distance from the embedding's centroid, and not the specific shape of the relevant distribution. And it is built from a separate tool in mathematics, which is the Laplacian eigenvectors. And that tool is not the entropy. Hence, it is not a fourth use of $H(\cdot)$. And also, it does not add a fourth point into the three above. In addition to that, Section 4.5.8 denotes that those two signals are not similar and identical. And one signal should not be substituted for the other one. These two are also not shown here to correlate with each other. Both signals can be in a relationship with the stability of the diffusion process output as well. Section 10 gives the relevant empirical testing process for that stability.

4.5 Geometric Paths, and a Real Time Two Dimensional Embedding of the Intent Graph

4.5.1 The Embedding as a Two Dimensional Web

Let $L = D - W_{\text{transmission}}$ be the similar graph Laplacian which is already defined in the section 4.1. From the eigenvectors of its Laplacian, a graph's structure can be recovered in a low dimensional form. Professor Miroslav Fiedler (refer [30]) researched and found this first. And that research is for the single eigenvector which is tied to a graph laplacian's lowest eigenvalue which is not a zero value. (this is also known as the algebraic connectivity) In addition to that, Professor Mikhail Belkin and Professor Partha Niyogi (refer [31]) built this idea further into a usable low dimensional embedding. They did this process by solving the generalized eigenproblem $L f = \lambda D f$ for the eigenvectors ψ_1, ψ_2 which are tied to the two smallest eigenvalues $\lambda_1 \leq \lambda_2$ which are not zero values. And in here, the less complex or more trivial and constant eigenvector at the eigenvalue 0 is intentionally excluded. And after that, this method gives each node u the fixed coordinate

$$(x_u, y_u) = (\psi_1(u), \psi_2(u)) \quad (37)$$

And this coordinate is computed again as the graph grows when new words appear.

A personal typing graph can be a graph which is not connected. Which means, separate and unrelated clusters of the vocabulary, which that human never types near one another, is a thing that can happen. And even a likely one. And also, the Equation (37) considers this graph as a connected graph. A graph which is not connected has a zero eigenvalue with the multiplicity which is equal to its number of connected components. And it is not one. Hence, this changes which eigenvectors are considered as the two smallest values which are not zero values.

This is the specific part which makes the "hexagon with the prey in the corners" spatial picture of this project true in a literal way for a one specific claim.

And it is not only an analogy. It is a node which its transmission edges spread evenly has a high $\varphi(u)$ value. (refer Equation no. (18)) Therefore, specific node is pulled by many nearby neighbor values in many directions. Hence, it should remain closer to the embedding's centroid. And also, a node which its transmission edges focus on a few amounts of next words has a low $\varphi(u)$ value. This node is pulled in one direction by a few strong edges. Hence, it should remain in a separate region area. Hence, the $\varphi(u)$ value should correlate with $\|(x_u, y_u) - \text{centroid}\|$.

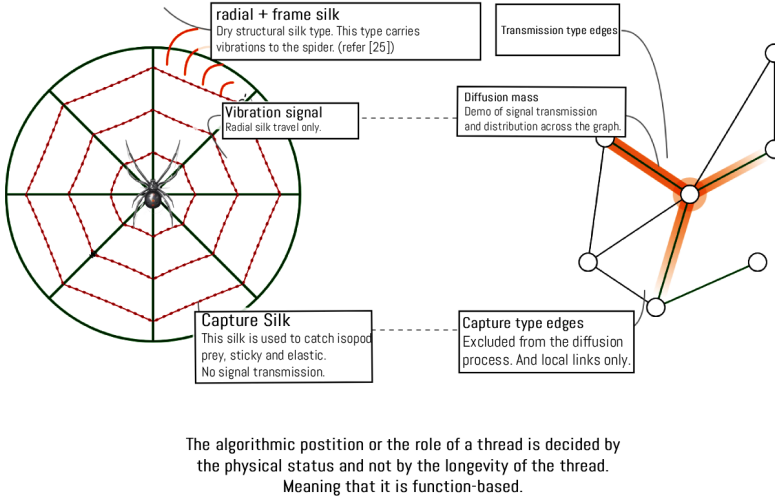


Figure 6. Illustration of the radial and frame structure of the web analogy and the corresponding algorithmic position of a thread or edge within the embedded intent graph.

4.5.2 Transforming a Sequence of Words into Paths

The section 4.5.1 gives a fixed coordinate for each node. In addition to that, a typed sentence gives a sequence of those next word distributions. In here, for the step i , define the position as the coefficient based average place of the diffusion output $\hat{p}_i$ across those coordinates.

$$\text{position}(i) = \sum_v \hat{p}_i(v) \cdot (x_v, y_v) \quad (38)$$

This is not only an analogy. $T = (\text{position}(1), \text{position}(2), \dots, \text{position}(n))$ creates a path through a plane. And there is a one point for each keypress. Meaning that, this process creates a curve through that plane.

4.5.3 Theories related to Heads and Plateaus of the Curve

For the step i , the speed value of the path is defined as

$$\text{speed}(i) = \|\text{position}(i) - \text{position}(i - 1)\| \quad (39)$$

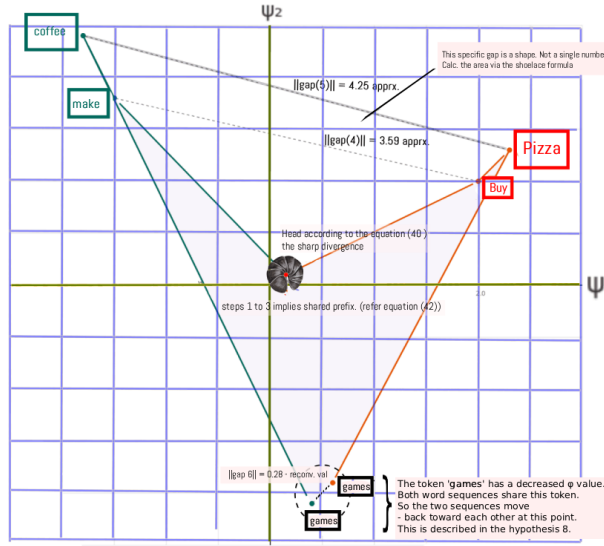


Figure 7. Illustrative two-dimensional trajectories produced by sequences of next-word distributions in the embedded intent graph.

A head is a local peak in the $speed(i)$ value. And this peak is greater than the prominence margin value θ_{head} . Meaning that, the path moves more faster into a specific region area.

$$head(i) = \begin{cases} 1 & \text{if } speed(i) \text{ is a local maximum with } speed(i) - \max(speed(i-1), speed(i+1)) \\ 0 & \text{otherwise} \end{cases} \quad (40)$$

This head is expected to be in a relationship with a word with a low $\phi(u)$ value entering the recent seed window. The hypothesis in the section 4.5.1 is applied in here to a one step. And it is not applied to the entire graph. In addition to that, a plateau is a run with minimum $n_{plateau}$ number of steps which happen one after the other. In that specific run, the $speed(i)$ value remains lower than the tolerance value $\epsilon_{plateau}$.

$$plateau(i) = \begin{cases} 1 & \text{if } speed(j) < \epsilon_{plateau} \text{ for all } j \in \{i, i-1, \dots, i-n_{plateau}+1\} \\ 0 & \text{otherwise} \end{cases} \quad (41)$$

And also, the plateaus are expected during a run of words with a high $\phi(u)$ value. Another scenario is when the path already remains near a well connected cluster. And the words after that only confirm it. In this method, the heads and plateaus picture receives a precise and a testable definition.

Honest status. θ_{head} , $\epsilon_{plateau}$, and $n_{plateau}$ are added to the free parameter list in Section 12. This paper does not decide the specific detection method used for the local peak. And it does not assign a measured value to the prominence margin value. proof/validation needs to be done - remove for now, address later

4.5.4 Two Paths in One Plane, the Shared Prefixes, and the Gap Between Them

If two sentences A and B share the exact similar word prefix with a length of k , their paths are the same over that prefix.

$$\text{position}_A(i) = \text{position}_B(i) \quad \text{for all } i \leq k \quad (42)$$

But this statement needs a stronger meaning than the words alone. This is because the section 6.9.3 introduces the $\alpha_{\text{effective}}(t)$ value in the equation no. (29). This is a parameter inside the diffusion process. And it is depending on the $\ell(t)$ value. This value gives for what amount of time the typing of that human remained in a sustained state. Meaning that, it is a function of the typing rate. And it is not a function of the word content. In that case, two sentences with the exact similar word prefix, which are typed at separate speeds or in separate hexarousal states, can include separate $\alpha_{\text{effective}}(t)$ values through that similar prefix. Hence, they can produce a different $\hat{p}_i$ value. And after that, a different $\text{position}(i)$ value. This happens even though the words are the same.

In addition to that, Equation (42) is correct as a statement about two possible continuations which separate from a one real and already typed prefix. Meaning that, both continuations receive the exact similar state until the point they separate. (the graph, the hexarousal baseline values, and the sustained state length) But for two separate sentences which are typed during two separate time periods, the "exact similar word prefix" needs this stronger meaning.

After the two sentences separate, the gap at the step i is defined as a two dimensional vector.

$$\text{gap}(i) = \text{position}_A(i) - \text{position}_B(i) \quad (43)$$

The sequence of these gap values creates a region area. And this region has an area value. It is not a one single number for the separation. This area can be calculated directly using the "shoelace formula" over the closed polygon created by both paths and their shared starting point. This is because a shape which opens and closes includes information which a one single number does not include. For an example, this depicts the way the two paths separate. (a wide and an early gap, compared to a narrow and a late gap which reaches the similar final distance) And it is not only the final distance value.

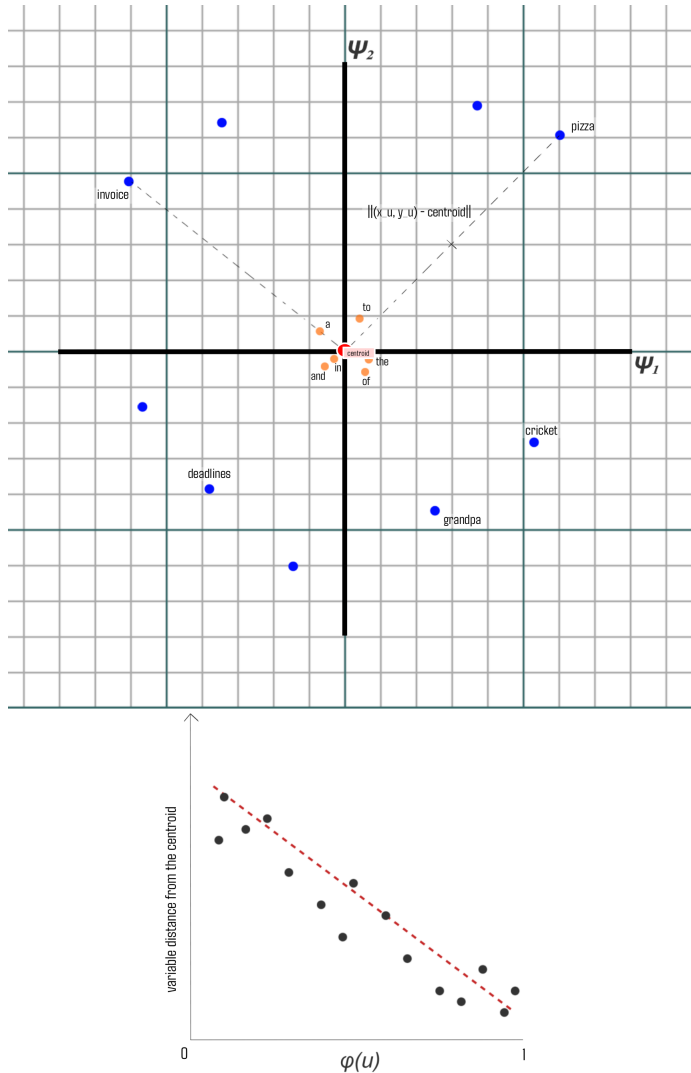


Figure 8. Illustrative two-path embedding and the corresponding gap behavior, showing how the separation between paths changes over the sequence.

4.5.5 The Reconvergence Process and Its Relationship With the Fan Dispersion Values

When both sentences share a word again later, whether the $\text{gap}(i)$ value closes back to the value 0 is depending on that shared word's own $\varphi(u)$ value. This is the most important point in this section. This is because it connects two previously separate parts of this paper into a one geometric explanation for a similar underlying idea. Those two parts are the spatial embedding in the section 4.5.1 and the fan dispersion coefficient process in the section 4.1.1.

1. A shared word with a high $\varphi(u)$ value has a broad and a generic connectivity. Meaning that, it pulls both paths in a lower amount, and only to the centroid of the embedding. Hence, this is a weak and a generic closeness. And it is

not a real reconvergence.

2. A shared word with a low $\varphi(u)$ value has a narrow and a specific connectivity. It pulls both paths strongly into the similar distinct cluster. Hence, this is a real and a valuable reconvergence.

This is also similar to the points made in the section 4.5.1 about theories for the single paths. And in here, it is applied to two paths. Meaning that, words with a low $\varphi(u)$ value carry and keep the position in a more strong way. Hence, a word with a low $\varphi(u)$ value, which is shared by two separated paths, becomes true for both paths in the similar place. And it closes the gap. A word with a high $\varphi(u)$ value carries the position in a lower amount. Due to that reason, sharing this type of a word closes the gap only in a small amount. Which means, only in the amount that any two separate points added to the similar centroid can close.

4.5.6 *The Relationship With the Already Existing Entropy or Confidence Signal*

The section 4.4 already gives a confidence signal to this architecture. This is $H(\hat{p})$, or the entropy value of the diffusion process output. And it is used directly in the necessity score in Equation (15). (refer section 7.3) In addition to that, the distance from the centroid (refer section 4.5.1) is a second and a geometry based signal. And it is calculated from the similar underlying object $\hat{p}$. These two signals should not be considered as a similar measurement in different types of forms. Which means that, those two are not similar and identical.

The entropy value measures how the probability mass is spread across the nodes. And this is separate from where those nodes are placed in the embedding. And also, the distance from the centroid measures where the probability coefficient based average place is. This is separate from how many nodes share that mass. Meaning that, a $\hat{p}$ value which is focused on a one node with a low $\varphi(u)$ value has a low entropy value. (which is a high confidence value in the section 4.4) And it is expected to remain more further from the centroid. (which is the hypothesis in the section 4.5.1) This can be given as the common scenario. And in this scenario, the two signals do not differ from each other.

But a $\hat{p}$ value which is spread evenly between two nodes in opposite directions from the centroid has a high entropy value. (which is a low confidence value in the section 4.4) And its centroid can remain close to the center of the embedding. It is not near either of those two nodes. Meaning that, it is the average value of two points which are approximately opposite and away from the center. And also, a $\hat{p}$ value which is spread evenly between two nodes which are near each other in the embedding has the similar high entropy value. But its centroid remains where both nodes already are. Which means, away from the center, and it is not near the center of the embedding. Due to that reason, the two signals can differ from each other. And this happens because of the design choices and processes. It is not a measurement error.

4.5.7 *A Worked Example With Illustrative Values*

Sentence A: "I want to buy pizza ... games." And also, Sentence B: "I want to make coffee ... games."

Step	word(s)	$position_A(i)$	$position_B(i)$	$\ gap(i)\ $
1-3	"I want to"	(0.15, 0.10)	(0.15, 0.10)	0 (exact, Eq. (42))
4	"buy" / "make"	(2.0, 1.0)	(-1.5, 1.8)	3.59
5	"pizza" / "coffee"	(2.3, 1.3)	(-1.8, 2.4)	4.25
6	"games" (shared, with a low $\varphi(u)$ value)	(0.6, -1.9)	(0.4, -2.1)	0.28

The gap value increases at the step 4. Meaning that, there is a head for both paths. (refer Eq. (40)) And the two paths separate into distinct and separate clusters. After that, this gap value increases more at the step 5. And after that, this value reduces in a large amount at the step 6. In that specific step, a shared word with a low $\varphi(u)$ value pulls both paths into the similar region area. Which means, this is the reconvergence process. And it is directly visible as a shape which is closing. (refer Section 4.5.5) And it is not calculated from a one number in the way the earlier version of this paper would have listed it.

4.5.8 Background of the Subject of Study

Component	Status
Theories and processes named as Spectral embedding through Laplacian eigenvectors, centroid, Euclidean distance, and shoelace formula area (Section 4.5.1, Equation no. (37)) $\varphi(u)$ predicts the distance from the centroid (Section 4.5.1)	Established mathematics [30, 31], applied in here to a new, live and a per user object
Centroid path for each sentence (Section 4.5.2, Eq. (38))	New and proposed point. Refer to the validation and experimentation section described later in this paper (refer Hypothesis 7)
Heads and plateaus as the formal peak and plateau detection process (Section 4.5.3, Eqs. (39)-(41))	New and proposed point. Refer to the validation and experimentation section described later in this paper
Shared prefix exactness (Section 4.5.4, Eq. (42))	Provable when the overall state is the same. (the graph, timing, and hexarousal history) And the weaker reading of "similar words only" is not exact
Gap as a shape between two paths (Section 4.5.4, Eq. (43))	New and proposed point. Refer to the validation and experimentation section described later in this paper
Reconvergence and the shared word's $\varphi(u)$ value (Section 4.5.5)	New and proposed point. Refer to the validation and experimentation section described later in this paper (refer Hypothesis 8)
Median line information content (Section 4.5.6, Eq. (44))	A question that is open and available. Refer to the validation and experimentation section described later in this paper
Reverse or retrodictive diffusion (Section 4.5.7)	A question that is open and available. Refer to the validation and experimentation section described later in this paper
Relationship with the entropy or confidence signal (Section 4.5.8)	The two signals are shown as able to differ from each other. And they are not shown to correlate or combine in a useful way

5. The Problem Related to the Blindness to the Valency

The architecture in Section 3 and Section 4 has a structural problem. This problem can be named as the blindness to the valency. equation (1), equation (16), and equation (17) do not include whether a selection event came from a text the human meant in a positive or a negative direction. The update process only includes that v was selected after u . And also, it includes whether that selection matched the output from the diffusion process. It does not include the type of the completed text.

A graph update process Φ on selection events (u, v, t) has the blindness to the valency if two selection events $e_1 = (u, v, t_1)$ and $e_2 = (u, v, t_2)$ come from texts with opposite emotional tone signals, and Φ produces the similar changes to (w, TF) .

Proposition No. (1) The update process in equation (1), equation (16), and equation (17) has the blindness to the valency.

Related Proofs equation (1) includes only $\mathcal{N}[\text{selection event at } t+\Delta t]$ and Δt . equation (16) includes only $\text{outcome}(u, v, t)$. This output result is defined by whether (u, v) carried the diffusion mass toward the next word the person selected. Which means that, this is a result related to the success of the prediction. It is not a result related to the emotional tone of the completed text. equation (17) includes only whether $TF(u, v, t)$ is greater than θ_{TF} . None of these three equations include another value which represents the surrounding completed text. Due to that reason, changing the positive or the negative direction of that text does not change the output of these three equations. ■

This proof follows directly from the definitions. And this is the important point in here. The blindness to the valency is included inside the update process. It is not a failure which appears later from another process. A testing process which does not create a mixed valency conflict test cannot identify this problem. (refer Section 10) equation (16) uses whether a prediction was confirmed. This does not include the emotional tone signal. Due to that reason, the processes in Section 6 are required for this problem.

5.1 An Example With Two Selection Events

Consider a human who selects the next word *pizza* two times during several hours. The first text is "I need to buy ____." The second text is "I have had enough ____." Both of them are selection events on an edge which ends at *pizza*. The first selection should strengthen a connection related to buying pizza. But the second selection should suppress that connection. Which means that, in the second text, the human is closing the intent rather than asking for more pizza.

Under equation (1), both events add the similar reinforcement value $+\eta$ to the relevant edges. Under equation (16) and equation (17), both events can increase $TF(u, v, t)$ toward the transmission type state when the diffusion process already produced that next word. The architecture does not include a process where the second event can add a negative or a suppression update. Confirming and closing texts can share the similar final next word in ordinary text. Requests, complaints, gratitude, and cancellations can be given as examples. Due to that reason, this is a main problem. It is not a problem which can be left for a later process.

5.2 Why the Existing Exhaustion Penalty Does Not Correct This Problem

The Tabu Search based exhaustion penalty in Section 3.2 can be considered for this problem. This process suppresses a node for a short period after a selection event. But this process does not separate the valency of those selection events. It suppresses each selection in the similar way.

The exhaustion penalty can stop the keyboard from suggesting the word the human selected a short time before. This is useful after "I need pizza" and after "I have had enough pizza." But equation (14) applies the similar short suppression period in those two states. And it does not change whether the relevant connection should be strengthened, weakened, or kept without another change for the future. Section 6 gives a process which changes the word connection. It does not only block the next word from being suggested again for a short period.

6. Proposed Extension: Valence-Gated Signed Diffusion

The Valence Gated Signed Diffusion part is a main invented contribution from this paper. This specific part includes several hypotheses created for the IntentSpider system. And also, it includes the relevant theoretical work in mathematics. These points are not presented as already verified points.

6.1 Design Goal

This process has three design goals. First, the process needs to run on every single keypress inside a phone. And it should not require a repeated connection to a server. Second, it should not require training data other than the data from that single user's same device. Third is a selection event should be able to push a word connection away from a next word. Which means that, the process should not only stop strengthening that connection. It should push the connection back when the surrounding context depicts that the human is closing the intent, rather than continuing that intent.

6.2 Scoring Valence On Device Through Typing interval patterns

Introduction. In spiders, especially the type "Wandering" Spider, the similar vibration event creates two behavioral responses to opposite sides, which is based on a physical property of the relevant signal. A low frequency vibration between 3 and 4 (used base values for simplification) produces the approach and a fighting or a threat response. And also, a greater vibration from approx. 350 to 450 value produces the escape response (wandering spider research paper. refer [26]). The spider does not need to read a meaning from that signal for this separation. Instead, the spider's CNS (or central nervous system) uses the physical shape of the signal. This same system is hypothesized in the IntentSpider's valence signal built from how the human typed a selection rather than the selection process output. Meaning it rejects the rule process. The disadvantage is that the timing signal can reduce the more clean precision a word list could provide when timing is a unclear or an unrelated signal for the human's intent, which carries a risk of being impractical with the daily life into a some extent. (refer the section no. 12).

Definition. For a selection event at time t , Δt_{avg} represents the average time interval between the keypresses in the previous context window. Δt_{ref} represents the baseline typing speed of that same user. This is tracked for each human separately, rather than using one fixed constant for all humans. In addition to that, κ is the correction rate value in that similar context window. Which means the simple number of backspaces per each key pressing event. This can be defined as a valence value.

$$val'(c_t) = \tanh(a \cdot (\Delta t_{ref} - \Delta t_{avg}) - b \cdot \kappa) \quad (20)$$

In this context, $val'(c_t)$ is a signed value. The sign of this specific value is the main part used by the processes from subsections 6.3 to subsection 6.5. Below in here, the subsection 6.9.1 adds a second axis from the similar typing rate signal. This second axis is unsigned, and it is named as arousal. Which means that, it represents a size and not a direction. Section 6.9.1 builds a further group of processes from this value without changing $val'(c_t)$.

6.3 Signed Edges

Every edge is extended to carry a signed value $s(u, v, t)$ between -1 and 1 , in addition to the existing raw coefficient value. For each selection event, this value updates as

$$s(u, v, t + \Delta t) = \text{clip}(s(u, v, t) \cdot e^{-\Delta t / \tau_v} + \eta \cdot \text{val}'(c_t) \cdot \mathbb{I}[\text{selection event}], -1, 1) \quad (7)$$

In here, c_t is the context window around the selection event at time t . And $\text{val}'(c_t)$ is the frequency based signal defined in the equation no. 20. (refer Section 6.2). A confirming selection has a $\text{val}'(c_t)$ value greater than 0. This pushes $s(u, v, \cdot)$ into a positive direction. A closing or rejecting selection has a $\text{val}'(c_t)$ value less than 0. This pushes the signed value into a negative direction, rather than adding the same positive increase from Equation (1). This is the direct correction for the failure mode proved in the main section no. 5. There was a pizza example, the two selection events now create updates with opposite signs. But this result depends on whether typing interval patterns is a useful signal for confirming or rejecting intent in a practical system. Section 6.2 states that limitation, and Section 10 gives the relevant experiment based testing process.

6.4 A Third, Suppressive Set, and How It Now Relates to Transmission Fidelity

The two axes are separate by construction. $\text{TF}(u, v, t)$ measures whether an edge reliably predicts the next word the human selects. Which means that, this value is about predictive success and it does not include emotional content (refer Proposition 1). $s(u, v, t)$ measures whether the selection on that edge happened with a positive willness or a negative willness, according to the behavioral timing process previously created. Due to that reason, an edge can have the transmission type and also build a negative valence value at that same time as well. This depicts a word connection which is predicted reliably and selected in a repeated way, but also selected with a reluctant behavioral signal. The pizza example in that previous section denotes this specific state. After “I have had enough pizza” is typed a sufficient amount of times, the diffusion process can, into a some extent reliably produce the *pizza* again. But the typing interval patterns can depict that the human is closing the intent rather than asking for more.

The suppression rule, stated on the valence axis. An edge which builds a sustained and strongly negative signed coefficient moves into a third set named as $E_{\text{suppressed}}$. In the diffusion process, this set is used as a direct smoothing function process (refer Section 6.5). After that, the relevant word is not removed from the graph. Condition for moving an edge into that set uses only $s(u, v, t)$. It is separate from the TF based role of the words and edges in here.

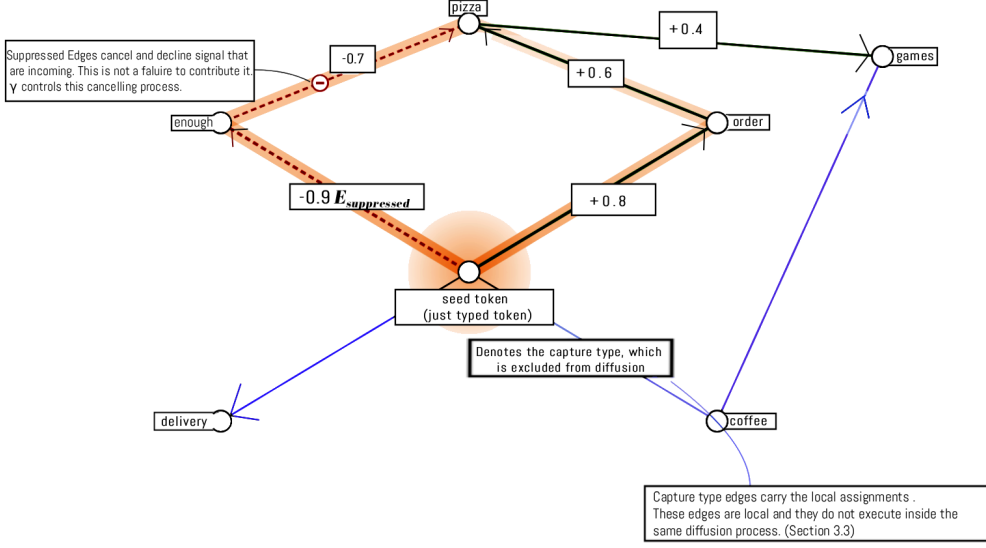


Figure 9. Illustration of transmission-type and capture-type edge roles, including the diffusion path and the direct non-forwarding association.

$$\int_{t-T'}^t \max(0, -s(u, v, \tau)) d\tau > \theta_s \implies (u, v) \text{ moves into } E_{suppressed} \quad (21)$$

Equation (21) is the ordinary route into $E_{suppressed}$. This equation adds the negative valence over a previous time window T' . Which means that, this is a slow building process. Section 6.9.4 below proposes a second and sudden route. In this route, one detected “shock” moves the edge into $E_{suppressed}$ immediately. It does not wait until the value passes θ_s . And this shock route uses a decay constant which is slower than the ordinary $E_{suppressed}$ membership. These two routes are not the same process. One reaches the set through a gradual buildup, and the other reaches that similar set through one sudden event. Section 6.9.4 explains why the motivating analogy includes those two speeds.

A tension from this integration which is not solved here. Sections 3.2 and 3.3 built the TF based typing process with not using ratchet. An edge can move between the transmission type and the capture type as the TF value increases or lowers. The $E_{suppressed}$ process has a different purpose. It is slow and persistent, so a rejected word connection does not return to the ordinary state immediately after the human stops rejecting it. Due to that reason, the architecture includes one process created for fast reversal and another process created for slow persistence. An edge can be typed quickly as transmission or capture, and suppressed slowly at the similar time. Section 6.9.4 adds a third speed into this structure. A detected shock creates an immediate entry into $E_{suppressed}$. This exists alongside the fast TF reversal and the gradual negative value buildup from Equation (21). Which means that, a reader following how the separate system parts change should track three speeds and not only two.

The $E_{suppressed}$ process and its relationship with TF based typing is the most unverified construction in this paper. Non of the CLS based continual learning architectures discussed in Section 2.2, DualNet, CLS ER, and ITDMS, use a

third inhibitory memory channel. Each of those architectures uses two memory processes. And non of them includes this specific mismatch between a reliability value that changes freely and a suppression state designed to remain for a longer period. Whether this third set improves calibration, or produces an instability, needs the testing process in Section 10. Further arguments in this paper cannot verify that point. We include this process because it directly addresses the design gap stated in Section 5. And also, removing the least verified part from a paper which identifies open questions would remove one of the main purposes of the paper. But Equation (21), $E_{suppressed}$, and its relationship with Equation (17) should be treated as the least verified construction in this document. This is important for any reader who considers this point as a main claim for a patent filing as well.

6.5 Signed Diffusion Read Out

A negative edge should actively cancel a spreading signal. It should not only fail to add more signal. Due to that reason, the unsigned transmission type Laplacian $L = D - W_{transmission}$ from Equation (4) is replaced with a signed Laplacian. This signed Laplacian uses $W_{transmission}$ and $E_{suppressed}$ together. The process follows the signed spectral graph theory from Professor Jerome Kunegis and others (refer [18]).

$$L_{signed} = D_{| \cdot |} - W_{signed}, \quad W_{signed} = W_{transmission} - \gamma \cdot W_{suppressed} \quad (9)$$

In this equation, $D_{| \cdot |}$ is the degree matrix built from the absolute edge coefficient values. This is required for the signed Laplacian. The γ value is greater than 0, and it controls how strongly a suppressed edge cancels the incoming signal at that node. The heat kernel and personalized PageRank read out processes from Section 4 remain available. Their seed comes from the fan dispersion construction in Section 4.1.1. In here, L_{signed} replaces L , and W_{signed} replaces $W_{transmission}$.

6.6 Algorithm

Algorithm 1. Function Typed and Valence Gated Diffusion Update and Suggestion Process for Each Selection Event

These are required - selection event (u, v, t), context window c_t , current edge set E where each edge includes w, TF, and s values, and the current $E_{suppressed}$ set.

1. The system calculates val' using $ScoreValence(c_t)$.
Notice - this process uses the typing interval patterns and the correction rate in Equation no. (20). (refer Section 6.2)
2. The system updates $s(u, v, t)$ using the val' value through Equation no. (7). (refer Section 6.3)
3. The system updates $w(u, v, t)$ through Equation no. (1).
Notice - this is the raw coefficient value. And this process does not use a channel branch. (refer Section 3.2)

4. The system updates $TF(u, v, t)$ through Equation no. (16).
Notice - this is the transmission fidelity value. And it uses the result of the current event, which is $outcome(u, v, t)$. (refer Section 3.3)
5. **if** the $TF(u, v, t)$ value is greater than θ_{TF} , **then** the system marks (u, v) as a transmission type edge. After that, it includes this specific edge in $W_{transmission}$ through Equation no. (17).
6. **otherwise**, the system marks (u, v) as a capture type edge. After that, it leaves this specific edge out of the propagation process through Equation no. (17).
7. **end if**
8. **if** the suppression condition in Equation no. (21) is true for $s(u, v, \cdot)$, **then**
9. The system moves (u, v) into $E_{suppressed}$. (refer Section 6.4)
Notice - this check happens as a separate process from steps 5 through 7. And it is not an otherwise branch.
10. **end if**
11. On the next keypress, the system uses the last k typed words.
12. The system sets $s(0)$ using $FanDispersionSeed(u_t, \dots, u_{t-k+1})$ through Equation no. (19).
Notice - the process in Section 4.1.1 limits this seed process to $W_{transmission}$.
13. The system calculates $\hat{p}$ using $LocalPush(s(0), W_{signed}, \alpha, \epsilon)$. (refer Section 4.3)
Notice - this process extends the local push process to signed coefficient values and an edge set which is not the same on each keypress. Those are two questions that are open and available in Sections 3.3 and 6.7.
14. **return** the top 3 words ranked using the $\hat{p}$ value. And also, return $H(\hat{p})$ as the uncertainty signal. (refer Section 4.4)

Step 8 is separate from steps 5 to 7. This is a intentional choice. The reason for this is, the suppression state condition and the transmission or capture type role are measured on two separate axes dimensions or axes planes and due to that reason one update can change an edge from capture type to transmission type in step 5, and also move that edge to the to execute the process suppression in step 8. This algorithm is not choosing a one result. But instead two results. The W_{signed} construction process in the subsection 6.5 decides how those results work as a combined version in the next step of the diffusion process.

6.7 The Local Push Mechanism State

Section 4.3 uses the local push algorithm from Professor Reid Andersen and others (citation no. [1]). Its $O(1/(\alpha\epsilon))$ runtime limiting value is proved for a graph

with coefficients which are not negative values. In that graph, the relevant matrix has a clear random walk process interpretation. The signed values in the equation no. (9) change that interpretation. And Probability mass is not kept in that equal way when a part of that value can be cancelled rather than only moved to another node.

This IntentSpider emotional engine is built on already published mathematics and research by other scientists. and this was the large point and the claim between this emotional engine processes and the runtime viability to run IntentSpider inside a mobile device (examples - iPhone, iPad). It is not the only one now. Section 3.3 adds a second and separate change from the original method by Professor Reid Andersen and others. The limit between the transmission type and capture type changes on every keypress. Section 4.3 explains that the signed values and the changing edge set add together, rather than replacing each other. A complete mathematical treatment needs to limit the residue related behavior under signed pushes on a graph where the relevant edge set changes at the similar time. As a one, this combined problem is complex and complex than each of those as separate ones.

6.8 Task Log Based on History of Word Acceptance to the System Using Coefficients in Input Contexts

6.8.1 Introduction

6.9 History Based Task Scheduling Using Coefficients From Previous Contexts

6.9.1

Section 4 ranks every next word by using its diffused mass value. This is p or $s(t)$ in Equations (5) and (4). But a separate choosing question remains after that process. The diffusion process can produce several next words at the similar time. Some of those words can include a already existing structure in the graph. This structure is named as "residue" or "residue values" in here. Another next word can be a new word and can result in a strong activation.

The system needs a process for selecting the most likely relevant word to suggest first. Like in a hierarchical model. The valence gate from Sections 6 through 6.5 does not make this decision. That process decides whether an edge becomes stronger or more weak across the timeframe. But it does not decide which one of the already active next words receives the first priority on the current keypress. And for that, this subsection gives a mathematical process for that choosing event.

6.9.2 Residue Value

The residue value is defined relative to θ_{TF} . A value of 0 represents a next word where the relevant edge has a transmission fidelity value of 0. A residue value which increases toward 1 represents an edge where the transmission fidelity increases toward the relevant transmission type margin value. For a next word v , suppose the system reaches it using an edge (u, v) . And suppose that edge already includes a transmission fidelity value (refer section 3.2). In here, the residue value is defined as

$$r(v) = \min\left(\frac{TF(u, v, t)}{\theta_{TF}}, 1\right) \in [0, 1] \quad (22)$$

In here, $r(v) = 0$ means that $TF(u, v, t)$ is 0 on the relevant edge. When $r(v)$ increases toward 1, the TF value also increases toward θ_{TF} . The $\min(\cdot, 1)$ process limits the residue value at 1. Once the edge (u, v) passes the transmission type margin value, it becomes a transmission type edge. After that, it competes in the diffusion ranking through its own values (refer section 4.4). At this point, it is not a residue word which competes with a fresh word through this specific choosing rule. Which means that, equation (22) is used before that edge becomes a transmission type edge.

6.9.3 The Support Value From Past Selection Events

The priority of a fresh next word is built from its previous selection events $e_1, \dots, e_n$. Each event includes a time value named as t_i . And also, each event includes a context window c_i . This process defines two values for each one of those events.

$$R_i = \exp\left(-\frac{t - t_i}{\tau_0}\right) \text{ (PR Factor)}, \quad S_i = \text{sim}(c_i, c_t) \in [0, 1] \text{ (PCS Factor)} \quad (11)$$

The PR Factor is known as the Pure Recent Factor, and the PCS Factor equals the Pure Context Similarity Factor. R_i represents the recency of the event. A recent event produces a greater R_i value. And an older event produces a lower R_i value according to τ_0 . S_i represents the value of similarity between the earlier context c_i and the current context c_t . Those two values are separate. And they are not directly added. Instead, the system combines them by using a logarithmic opinion pool. This is also named as a geometric opinion pool. It is an already existing process for combining two information sources through a one interpolation value. Professor C. Genest and others introduced this method (refer [7]). Another property of this process keeps the result consistent when the form of the parameters changes (refer [8]). This is different from the linear or the additive options.

$$w_i(\rho) = R_i^{1-\rho} \cdot S_i^\rho, \quad \rho \in [0, 1], \quad \text{Support}_\rho(v, t) = \sum_i w_i(\rho) \quad (12)$$

A less complex process would be able to scale the recency constant directly by the value of context similarity. For an example, $\tau_{eff} = \tau_0(1 + \beta \cdot \text{sim})$. But this process does not produce the relevant end value. When τ_0 increases without a limit, each event's decay value increases toward 1. This happens without depending on its own value of similarity. Which means that, the pure context end value becomes an event count with equal coefficients. It does not become a context based value. Due to that reason, equation no. (12) uses the logarithmic pool. Its two end values can be checked directly. The two points below include those checks.

6.9.4 The Two End Values of the Interpolation Process

The first end value. $\text{Support}_0(v, t) = \sum_i R_i$. This is the Pure Recent value. And it does not depend on the context value.

Checking process. Put $\rho = 0$ into equation (12). Then $w_i(0) = R_i^1 \cdot S_i^0 = R_i$. ■

The second end value. $\text{Support}_1(v, t) = \sum_i S_i$. This is the Pure Context value. And it does not depend on the recent value.

Checking process. Put $\rho = 1$ into equation (12). Then $w_i(1) = R_i^0 \cdot S_i^1 = S_i$. ■

Those two end values were also checked by using numerical values. Which means that, they were not left only as algebra. The checking process used two created selection events. Event A is recent. In this event, t minus $t_i = 1$. But it has a lower context relationship, where $S_i = 0.10$. Event B is old. In this event, t minus $t_i = 50$. But it has a stronger context relationship, where $S_i = 0.90$. The process uses $\tau_0 = 10$. At $\rho = 0$, the Pure Recent total is 0.911575. This is exactly equal to $\sum_i R_i$. At $\rho = 1$, the Pure Context total is 1. This is exactly equal to $\sum_i S_i$. And these numerical values confirm the two exact end values.

Table 1 depicts another point which is not clear before checking the values between those two ends. At $\rho = 0.5$, Event A continues to have the greater contribution. At $\rho = 0.75$, Event B has the greater contribution. Which means that, the crossover happens between $\rho = 0.5$ and $\rho = 0.75$. In this specific example, the recent event needs a more lower coefficient before the context value receives the priority. This behavior comes from the geometric mean process. It is not a separate tuning decision. Due to that reason, $\rho = 0.5$ should not be read as an equal 50 and 50 balance in the practical system process. It continues to assign the priority to the recent event when the context relationship is not meeting the strength to the required level.

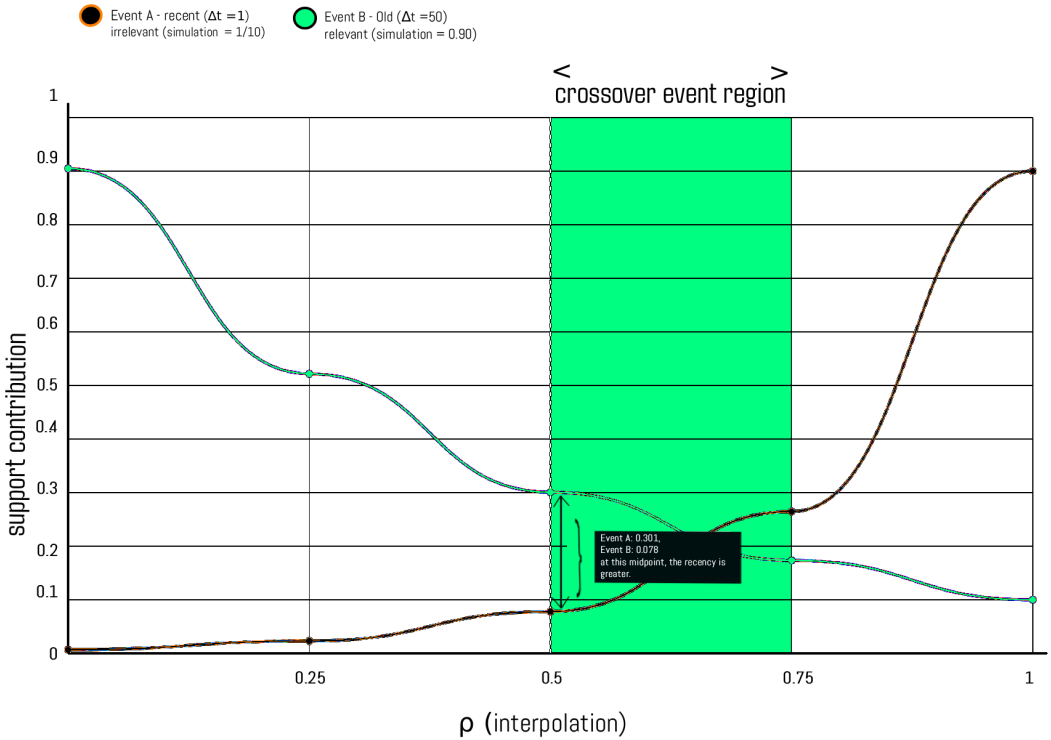


Figure 10. Illustration of the crossover between recent-event and context-based support contributions as the interpolation parameter ρ changes.

Table 1. The contribution from each event to Support_ρ when ρ increases from the Pure Recent value ($\rho = 0$) to the Pure Context value ($\rho = 1$). The table uses the two created events above. The crossover between $\rho = 0.5$ and $\rho = 0.75$ does not happen at the midpoint.

ρ	Event A contribution	Event B contribution	Selected winner contribution
0.00	0.90484	0.00674	A
0.25	0.52171	0.02291	A
0.50	0.30081	0.07787	A
0.75	0.17344	0.26474	B
1.00	0.10000	0.90000	B

6.9.5 Selecting Rules in between a Fresh Word and a Residue Word

A fresh next word v_{fresh} receives the priority over a residue next word v_{residue} iff,

$$\text{Support}_\rho(v_{\text{fresh}}, t) > r(v_{\text{residue}}) \quad (13)$$

Support_ρ and $r(v)$, which is defined in equation (22), use two different value ranges. In mathematical notations, those ranges are $[0, n]$ and $[0, 1]$. A normal-

ization value or a calibration value between those ranges is not defined in here as well. Due to that reason, the two ranges cannot be directly compared before that value is assigned. So, this can be considered as a freely available parameter. And section 6.8.6 denotes this point without assigning a value which was not tested.

6.9.6 Note About Already Established Research Knowledge

The logarithmic opinion pooling process is a already established method. But its use for the choosing process between the recent value and the context value in an on device personal intent graph was not found in the researches used for this paper. Due to that reason, this specific use is experimental and can be considered as a least verified part in this subsection.

1. The normalization value between the Support_ρ value range and the $r(v)$ value range is not defined. Section 10 includes the experiment based testing process for calculating this value.
2. ρ is a new free parameter. It is added to θ_{TF} , η_{TF} , τ_v , τ_{seed} , η , α , ϵ , and γ . These are experimental values. That correlation nearby the mid point does not change in a single line. So the manual tuning process for ρ variable is more complex than an ordinary single line mixing coefficient. The last point is that, whether these additions and changes reduce or improve the overall accuracy is a question. This requires experimental evidence from real human usage for months or even years to be completed. In a future IntentSpider version, these experimental results or an alternative will be included. It has the similar disadvantage denoted in section 6.4, in cases where the process of assigning priority to a next word with an already existing residue may reduce the priority of a newly added word without an earlier selection history. This section is well addressed, backed and supported in Section 4.

6.10 Details Related to Typing Rate Diff Values, Shared Cluster Reinforcement Process, Growing Radius Value, and Sudden Termination Process

In this section, these six processes came from the motivation by an attack and struggle analogy. Another special note is that they can be considered as invented mechanism as well. This analogy of attacking and struggling of an Orb Spider and Isopod preys inspired these ideas. This is a mathematical hypothesis built from this analogy.

The affective axis in subsection 6.9.1 and the communication channel point in the 6.9.8 are different from those six processes. Professor James Russell's research gives "valence" and "arousal" (These are described as valance and arousal axis and copied here as is.) as two dimensions of affect. As said previously, the term name "arousal" is used in the InterSpider system because it is the name of Professor Russell's second axis of his research. IntentSpider research's name for this second axis as the affective axis is named as "hexarousal" to avoid confusion, and etc. And Professor Joseph's research produces the communication channel difference previously discussed in the previous Section 2.8. (refer citation [29]) But Equation (23) only measures the delta of the typing rate, or the typing rate difference compared to the ordinary typing rate of that specific human. In a psychological perspective it also does not transform that

typing rate into a psychological measurement value. IntentSpider Webnet's input is a typing behavior based value and it is a different point when compared to other researches.

6.10.1 The Typing Rate Difference Signal

Section 6.2 gives the valence signal $val'(c_t)$. This signal gives whether the update process treats a specific selection as a confirmation result or a rejection result. But it does not give the amount that the current typing rate differs from the ordinary typing rate of that human. For an example, a slow reject and a faster reject with several corrections can both give a negative direction. The direction can be similar. But the amount of the typing rate change can be different. The correction rate is already denoted with κ in the (20) equation as well. The typing rate difference does not have a separate value in that equation.

Professor James Russell's circumplex model denotes valence and arousal as two separate dimensions (refer section 2.8 and citation no. [28]). The first dimension gives the pleasant to unpleasant direction. The second one gives the activated state to deactivated state dimension. As scientists conducted research related to this two dimensional method earlier, this foundation is not invented or novel. But a typing rate value does not become a measuring unit of Professor Russell's axis related to the arousal values, because the value increases. IntentSpider names this second system axis as hexarousal axis. And it names the value as noted below according to the typing data, which creates that axis overtime.

Let $x(t)$ represent the typing rate calculated through a short trailing window ending at t . And let $\mu_{user}(t)$ and $\sigma_{user}(t)$ represent the rolling mean and rolling standard deviation of $x(t)$ for that specific human. The system calculates these two baseline values from the earlier typing data of that human. It does not include the current window when calculating them. Which means that, the current difference cannot change its own baseline. In here, define

$$TRD(t) = \text{clip}\left(\frac{|x(t) - \mu_{user}(t)|}{\max(\sigma_{user}(t), \epsilon_{rate})}, 0, \text{cap}\right) \quad (23)$$

$TRD(t)$ type of notations refer to the standardized typing rate difference. ϵ_{rate} represents a small positive value. This positive value is important. It has two purposes. First, it is the value that prevents dividing by 0, which results "undefined x y" errors in C++. and second is when the earlier typing rate has no measured difference across its values. Equation (23) uses an absolute modulus difference. Due to that reason, $TRD(t)$ has no positive or negative directions. A period faster than the baseline level and a period slower than the baseline level can give a similar $TRD(t)$ value. The sign of $x(t) - \mu_{user}(t)$ keeps the faster or slower typing direction in places where another process is actively requesting it (or using it). And the sign of $val'(c_t)$ keeps the separate confirmation or rejection direction and it is used by the signed update process.

This separation is required. $TRD(t)$ does not prove a psychological state of the human. But rather, it is distance the current typing rate is from the ordinary typing rate of that specific human. Corrections, pauses, a device change, a physical interruption, or a deliberate change in typing speed can create that difference. For further details, in the subsection 6.9.6 denotes this loss of information. It keeps the separate typing direction inside a larger feature vector.

Δt_{ref} in Section 6.2 and the $\mu_{user}(t)$ and $\sigma_{user}(t)$ values in this section follow a similar per user structure. Non of them use a population typing margin value. A population value can read one human incorrectly when the ordinary typing rate of that human differs from the population value. IntentSpider operates inside a single device for a single human without any external servers (This was previously discussed in depth. Refer to section 3.1). Due to that reason, this baseline value is relevant to this design of the system.

6.10.2 Compounding Cluster Reinforcement Process Under Repeated and Prolonged activity

Introduction Equation (1) reinforces one edge after each selection event. But several other next word edges can remain in the active state during a similar short period. The motivation in here is the analogy of a spider receiving continuous and multiple vibration signals from a struggling prey across several threads, rather than receiving one separate strike. Here in IntentSpider system that specific analogy and process becomes a shared and clustered reinforcement process across the edges (connections between words) which were active recently.

Definition. Let $P(t)$ be the *active prey set*. This is the set of next word edges which received a selection or a defined partial activation during the short trailing window w_{active} before t . In this system, we use the already selected diffusion activation margin value value for a partial activity start function. A partial activation does not mean or imply that every diffusion value greater than 0. In that case, $P(t)$ can include a large part of the reachable graph.

$$P(t) = \{(u, v) : \text{lastactivation that is greater than or equal to the specific limit of } (u, v) \text{ occurs within } w_{active} \text{ before } t\} \quad (24)$$

With the number of edges in $P(t)$, the diffusion budget amount increases. Which means this is a dynamic value.

$$B(t) = \min(\eta_{cluster} \cdot (1 + \delta)^{|P(t)|-1}, B_{max}) \quad (25)$$

In here, $\eta_{cluster} > 0$ represents the base reinforcement value. $\delta \geq 0$ represents the increase added using the number of active edges. And B_{max} is a value that is assigned to the maximum value for the diffusion budget. When $P(t)$ includes one edge, the budget is $\eta_{cluster}$. When the number of edges increases, the budget also increases until it reaches B_{max} . Which means the equation no (25) allocates a greater reinforcement during a period with several qualifying active edges.

The system shares this budget across $P(t)$. Each edge receives one part based on its diffusion mass with values which are not negative values.

$$\Delta w(u, v, t) = \frac{B(t) \cdot \hat{p}(v)}{\sum_{(u', v') \in P(t)} \hat{p}(v')}, \quad \text{for each } (u, v) \in P(t) \quad (26)$$

Equation (26) needs a denominator value greater than 0. If the total diffusion mass across $P(t)$ is 0, then the system does not apply the shared update at that event. This specific update is added to the reinforcement or further strengthening update, which is in the first equation. (refer Equation (1)), And importantly, it does not change that update.

This is a small note about notations in this section.

1. $\eta_{cluster}$ used for base reinforcement value in equation (25) This is a separate symbol than β_1 , β_2 , and β_3 in the section 7,3 for necessity gating functions.
2. this is different than η , which the equation no. (1) using as learning rate. This should not be confused.
3. $\eta_{cluster}$ is similar to the pattern η_{TF} in the subsection 3.2.
4. The δ and B_{max} symbols are separate from other values in the paper as well.

6.10.3 Growing Engagement Radius Under a Sustained Typing Rate Difference

The motivation in here is the analogy of a spider changing its reach settings when the isopod in the default mode. In IntentSpider, this becomes a gradually increasing scenario in the diffusion distance while a more greater typing rate difference is already active. This is the rising magnitude part of the affective axis or hexarousal. It is a value which moves to more upper levels when the human's typing is in a repeated and a prolonged activity. This event listens and executes TRD(t). Which means that this process identifies the typing behavior of the human into a some extent, but, It does not necessarily imply IntentSpider identified a neurological or a psychological information of the human.

Definition. Let t_i represent the time of the current keypress. Let θ_D represent the typing rate difference margin value. And let $\ell(t_i)$ represent the number of consecutive keypresss ending at t_i where TRD remains greater than θ_D .

$$\ell(t_i) = \max\{n : \text{TRD}(t_{i-j}) > \theta_D \text{ for every } j = 0, \dots, n-1\} \quad (27)$$

This definition uses the actual keypress times t_i . It does not subtract an integer from a continuous time value. After that, the following function gives the gradual increase from this sustained period length.

$$\zeta(\ell) = 1 - e^{-\ell/L_0}, \quad \zeta(\ell) \in [0, 1) \quad (28)$$

The function starts from 0. It moves to 1 when the number of consecutive keypresss increases. But it does not reach 1 for a finite ℓ value. The effective teleport value used in Eq. (5) then becomes

$$\alpha_{effective}(t_i) = \alpha_{base} \cdot (1 - \zeta(\ell(t_i))) \quad (29)$$

When they click on keyboard, for each stroke, IntentSpider uses $\alpha_{effective}(t_i)$ in equation no. (5) instead of the ordinary α value. α_{base} represents the α value used when the sustained typing rate difference period is not initiated.

There is also a collision with the notation ρ . In the equation no. (12) in subsection 6.8.3, it already uses ρ for the value of interpolation between the recent value and context value, which are variable values in the logarithmic geometric opinion pool. They should not be confused. Due to that reason, this paper uses $\zeta(\ell)$ for the Equation no. (28). And it uses θ_D instead of θ_A . This is because the executor is the typing rate difference value and not a direct derivation from Professor Russell's arousal axis theories. They should not be confused as well.

The α value controls how frequently the diffusion returns to the starting point (refer Section 4.2). A lower α value lets the signal travel further before that return. When $\ell(t_i)$ increases, $\zeta(\ell)$ increases and $\alpha_{effective}(t_i)$ reduces. Which means that, the diffusion distance increases gradually while the greater typing

rate difference is continuing. Hence the direction of the equation matches the growing radius value.

In the equation (29), this also changes $\alpha_{effective}(t_i)$ more closer to 0 when the sustained period becomes longer. The processing amount in the $O(1/(\alpha\epsilon))$ bound increases when the α value reduces. This is the specific disadvantage created by this process. And this disadvantage appears during the same period where the system needs to answer in each of those keypresss. We also add the human typing behavior as an input to the diffusion process. The shared prefix proposition in Section 4.5.4 considers the words and the graph as the inputs. In the equation (29), it adds the history of the keypress timing. As in the section 4.5.4, due to that reason, two prefixes with the same words are not allocated to output a similar diffusion output when a human typed those words through different different timing patterns.

Section 4.3 keeps two questions about the $O(1/(\alpha\epsilon))$ bound. The first one comes from signed edge values. The second one comes because the edges in the transmission type edge set are not the same on each keypress. Those two points ask whether local push finalizes and ends in a fixed amount of time under those changed graph conditions. And the process in this subsection adds a third and separate type of disadvantage. It does not change the α value into an unusable value during one finite keypress period. Instead, it deliberately reduces the α value used by the bound. And the processing amount increases when that value reduces. Due to that reason, this is a processing disadvantage. And this disadvantage is denoted in the section 9.2 as well.

Honest status. θ_A and L_0 are added to the free parameter list in Section 12. The paper does not decide whether $\alpha_{effective}$ needs a minimum value. Such a value would stop it from moving close to 0. But it would also limit distance a sustained hexarousal period can extend the diffusion. proof/validation needs to be done – remove for now, address later

6.10.4 The Sudden Negative Selection Responses

The motivation in here is the analogy of a physically struggling isopod prey, which makes the spider travel from one specific point to the other and remain within that point for a short period of time. The first response is related to the specific edge connected with that event. The second response changes a specific value in the system for a limited amount of time. Here in IntentSpider system, this specific analogy and process transforms into a pattern taken from $val'(c_t)$, which gives whether the update process reads a specific selection as a confirmation result or a rejection result, and the correction rate, which is already denoted with κ in the (20) equation. But the detected event does not prove that the human experienced a psychological shock. It only gives that pattern from those two values, and it does not necessarily imply IntentSpider identified a neurological or a psychological information of the human.

Let $\mu_{val,user}(t)$ and $\sigma_{val,user}(t)$ represent the rolling mean and standard deviation of the earlier $val'(c_t)$ values for that specific human. The system does not include the current value in this baseline. Let $\epsilon_{val} > 0$ represent a small positive value. This value prevents the standard deviation from becoming 0 in the margin value calculation. The system detects the event when $val'(c_t)$ is lower than the earlier mean of that human by more than k standard deviations

$$\text{shock}(t) = \begin{cases} 1 & \text{if } \text{val}'(c_t) < \mu_{\text{val},\text{user}}(t) - k \cdot \max(\sigma_{\text{val},\text{user}}(t), \epsilon_{\text{val}}) \\ & \text{and } \kappa(t) > \kappa_{\text{burst_marginvalue}} \\ 0 & \text{otherwise} \end{cases} \quad (30)$$

Equation (30) detects an unusual negative value in the statistical values of that human, together with a correction burst. It does not identify the reason which created that pattern. An ordinary typing error, a physical interruption, a device change, or typing with frustration can activate it. Hence, the equation gives an sudden negative selection response inside the system.

Local response. When $\text{shock}(t)$ equals 1 during a selection event on edge (u, v) , the system moves that specific edge into $E_{\text{suppressed}}$ immediately. It does not wait until the negative integral in Eq. (21) passes θ_s .

$$\text{shock}(t) = 1 \text{ on } (u, v) \implies (u, v) \in E_{\text{suppressed}}, \text{ with decay constant } \tau_{\text{shock}} \quad (31)$$

In here, τ_{shock} represents the persistence period for this route into $E_{\text{suppressed}}$. A greater decay constant gives a slower reduction when suppression uses the exponential decay form. Section 6.4 does not give a separate symbol to the ordinary $E_{\text{suppressed}}$ decay constant. Due to that reason, this subsection can state that τ_{shock} is intended to be greater than that ordinary constant. But it cannot give a numerical relationship before the ordinary constant receives a value. Equation (31) creates a longer suppression persistence. It does not create a deeper suppression amount unless the implementation adds a separate suppression magnitude.

During a cool down window w_{shock} after the detected event, the system increases the necessity gating margin value in Eq. (15). Section 7.3 owns that margin value. And it gives this temporary change as Eq. (36).

This corrects the earlier equation direction. Equation (15) displays a suggestion when $N(w, t)$ is greater than κ_N . Reducing κ_N makes that requirement easier to pass. Which means that, the system can display more suggestions. A temporary response which is more careful needs the opposite direction. Due to that reason, the system increases κ_N . This makes the requirement more difficult to pass.

This is a small note about notations in this section.

1. This margin value is κ_N . It is not τ_N .
2. The τ symbols in this paper represent decay constants. This includes τ_v , τ_x , τ_0 , and τ_{seed} .

3. The necessity margin value is not a decay constant. This subsection describes the relationship between the necessity gating algorithm and the overriding process. The local response and the system wide response give a specific form to an earlier design point. That design point was a second signal which can return or cancel a diffusion output with a greater confidence from being displayed. The necessity gate in Section 7 already gives the continuing event executor by using the process denoted there. In addition to that, equation no. (30) adds a separate event event executor. The system does not add the continuing gate and the sudden executor into a one score. The event changes the already existing margin value for a limited amount of time. Then the equation no. (31) is executed to change that edges' state

Honest status. k , $\kappa_{burst_marginvalue}$, w_{shock} , τ_{shock} , and $\Delta\kappa_{shock}$ from Section 7.3 are added to the free parameter list in Section 12. This paper has not tested whether Eq. (30) separates the intended shock events from ordinary frustrated typing. An ordinary period of correction activity may also activate it without carrying the special meaning intended here. proof/validation needs to be done – remove for now, address later

6.10.5 The Timeout Process of the Grace Period and the Settling Event (the function of returning to normal state).

This specific mechanism is inspired by a behavior of a animal predator (Orb Spider). In this scenario, the elevated responsiveness factor reduces at the moment when the pray isopod stops their physical activity. Which means in this system, the "active period" gets terminated if new prey are not greater than or equal to the relevant margin value within that set timeframe. Once the active period state ends, both the temporary reinforcement and the temporary radius difference reduce and decay again to their previous base values. As a notable factor, this specific decay rate is fast if the active period state stops with an already confirmed selection than it becomes invalid after that timeframe expiration. (or timer expiration)

The definition of this process is, if q represents a single period of activity, then if t_q^{end} denote the time of its last and final activation that is greater than or equal to the specific limit A settling event starts when none of the edges in $P(t)$ receives another activation that is greater than or equal to that limit during T_{grace} . The default value used in here is 60 seconds, or a complete minute.

Let $\Delta w_q(u, v)$ represent only the coefficient change added by the equation no. (26) during the period, which is q .

And let $\Delta\alpha_q(t') = \alpha_{base} - \alpha_{effective}(t')$ represent only the teleport reduction added by Eq. (29) during that period. The system keeps these values separately. This is because it cannot reduce the changes from one specific period later if it has already combined them permanently with the ordinary updates.

A confirmed period is one where $val'(c_t)$ is greater than 0 at the final selection event. Otherwise, the period is unconfirmed. Define $m_q = 1$ for a confirmed period. And define $m_q = m_{unconf} > 1$ for an unconfirmed period.

For $t \geq t_q^{end} + T_{grace}$, the return function process is

$$\begin{aligned}\Delta w_q(u, v, t) &= \Delta w_q(u, v, t_q^{end}) \cdot e^{-(t-t_q^{end}-T_{grace})m_q/\tau_v}, \\ \Delta\alpha_q(t) &= \Delta\alpha_q(t_q^{end}) \cdot e^{-(t-t_q^{end}-T_{grace})m_q/\tau_\alpha}.\end{aligned}\tag{32}$$

The first line, after executing the function for it, it returns the reinforcement added during that specific period to 0. The second line returns $\alpha_{effective}$ to α_{base} . This is executed by reducing the "teleporting" difference added during that period. τ_v and τ_α keep the coefficient return and the radius return as two separate speeds. And for an unconfirmed period, this uses $m_{unconf} > 1$. Which means that, its temporary changes reduce faster than the temporary changes from a confirmed period.

In this process as obvious, this requires the system to store these values and update them. Including keeping the update source, the related activity period, the final event, and the two temporary values separately. Without that information values, the equation (32) cannot find which output taken from Equation

(26), which output taken from Equation (29), and which output relates to the normal system update.

Honest status. This is not only an equation. It also creates a bookkeeping requirement. The implementation has to find which exact coefficient changes and which exact radius changes came from one specific period after that period has already ended. Then it has to decay those changes at a separate rate from the ordinary background decay. This gives a nontrivial engineering disadvantage. This paper keeps that point instead of removing it. T_{grace} and m_{unconf} are added to the free parameter list in Section 12. proof/validation needs to be done – remove for now, address later

6.10.6 Discovered Per User Typing Sub State Clustering Process”

Introduction. TRD(t) is a one single value and it has no direction. Meaning that, its a scalar quantity. It does not keep whether the typing becomes more faster or more slower. And a similar value can appear with a different correction rate, pause structure, or $val'(c_t)$ direction. For an example, fast typing with a lower number of corrections, fast typing with greater number of corrections, and a long paused state after a selection are separate typing states. Equation (23) can place them under a similar magnitude. After that, we keep those measured patterns as typing sub states as well.

Let

$$f(t) = (z_x(t), TRD(t), \tilde{\kappa}(t), g_{pause}(t), val'(c_t)), \quad z_x(t) = \frac{x(t) - \mu_{user}(t)}{\max(\sigma_{user}(t), \epsilon_{rate})} \quad (33)$$

In here, $z_x(t)$ keeps the faster or slower typing direction. And TRD(t) keeps the limited absolute magnitude from Equation (23). $\tilde{\kappa}(t)$ represents the correction rate after a per user normalization. $g_{pause}(t)$ represents the pause pattern feature. And $val'(c_t)$ is the selection direction already used in the previous section no. 6.2. These separate parts need comparable scaling before one distance margin value combines them. Otherwise, one part with a greater numeric unit can decide the cluster distance by itself.

An online clustering process can run per user and inside the device IntentSpider is running. It groups the recent $f(t)$ values using a distance margin value $\delta_{cluster}$. (The number of groups is not a fixed value before the typing data exists). And the meanings of those groups are also limited to the patterns in the measured typing features. But this equation (33) is not a definitive output of the human user’s emotional state, and it alone cannot name one group as happy state, sad state, rushed state, frustrations or another emotional state.

Equation (33) does not decide one specific method for $g_{pause}(t)$, the normalization used on $\tilde{\kappa}(t)$, the distance margin value, or the clustering process. This note is for denoting them.

$g_{pause}(t)$ represents the pause pattern feature. $\tilde{\kappa}(t)$ represents the correction rate after a per user normalization. These separate parts need comparable scaling before one distance margin value combines them. Otherwise, one part with a greater numeric unit can decide the cluster distance by itself. An online clustering process can run per user and inside the device IntentSpider is running. It groups the recent $f(t)$ values using a distance margin value $\delta_{cluster}$. (The number of groups is not a fixed value before the typing data is available.)

6.10.7 The Process of Combining Several Typing Vector Values or Signals Into One Value

Sections 6.9.2 and 6.9.3 use a typing rate difference without a direction. Equation (33) keeps more than one measured behavior. This subsection gives a one bounded value from those features for a place where the system needs a multiplier with values which are not negative values. This value is a behavioral interaction coefficient. It is not a direct measurement of a psychological state.

Let $\tilde{g}_{pause}(t)$ represent a pause feature normalized into a value from 0 to 1. Let $\tilde{\kappa}(t)$ also remain from 0 to 1. And let $\widetilde{TRD}(t) = TRD(t)/\text{cap}$ when cap is greater than 0. In here, define

$$\xi(t) = \text{clip}\left(\tanh(\nu_1 \widetilde{TRD}(t) + \nu_2 \tilde{\kappa}(t) + \nu_3 \tilde{g}_{pause}(t) + \nu_4 |\text{val}'(c_t)|), 0, 1\right) \quad (34)$$

The ν_1 to ν_4 coefficients have values which are not negative values. Equation (34) therefore gives 0 when all four input magnitudes are 0. And it moves to 1 when their coefficient based combination increases. The absolute value of $\text{val}'(c_t)$ keeps the confirmation or rejection direction outside this magnitude. This is required if $\xi(t)$ replaces another multiplier which cannot use a negative value. If the system uses $\text{val}'(c_t)$ with its sign inside the sum, a negative selection can reduce or reverse the coefficient. That would create a separate process.

This is a note about notations in this section.

1. This value is $\xi(t)$. It is not $\alpha_{coefficient}(t)$. The α symbols in this paper represent the teleporting process. This includes α , α_{base} , and $\alpha_{effective}$. They should not be confused and please note that we use $\xi(t)$ for the equation (34).

2. There is a collision with the notation w . This paper uses $w(u,v,t)$ for edge coefficients. And in the equation no. (12), it already uses $w_i(\rho)$ for the logarithmic pool. Section 7.3 uses β_1 , β_2 , and β_3 for necessity gating functions as well. They should not be confused. And we use ν_1 to ν_4 for the coefficients in the equation (34).

The earlier coefficient names w_1 to w_4 also connect with notation already used in this paper. This paper uses $w(u,v,t)$ for edge coefficients. And the equation no. (12) uses $w_i(\rho)$ for the logarithmic pool. And section 7.3 uses β_1 , β_2 , and β_3 for necessity gating algorithm.

In addition, separately, coefficients in Eq. (34) are named ν_1 to ν_4 .

Equation no. (27) uses $TRD(t)$ value to find a period. Equation (25) uses the number of currently active edges. In that case, any of those equations do not use $\xi(t)$. This should not be confused. $\xi(t)$ is defined as a separate and a combined signal for places in this system which need a single value that stays within a fixed range, with values which are not negative values.

6.10.8 Scaling Process of the Communication Channel

The difference between the communication channels does not come from the analogy of the spider. Professor Joseph Walther's "Hyperpersonal Model" (refer [29]), together with the cues filtered out and social information processing research discussed in Section 2.8, showed that computer mediated communication is different from face to face communication. And this research establishes something very specific. Meaning that, how the affective information appears and how it is read is depending on the specific medium of the communication

used. Text messaging, email, note taking, and another written communication system do not give the human the same typing conditions.

The implication for this system is a limited one. A typing rate baseline calculated inside one communication channel should not be used directly for another communication channel. The ordinary typing rate and the pause behavior of the same human can change based on the task and the communication system. Let $\mu_{user,channel}(t)$ and $\sigma_{user,channel}(t)$ represent the earlier baseline values calculated for one human inside one communication channel. One illustrative calibrated value is

$$TRD_{channel}(t) = c_{channel} \cdot \text{clip}\left(\frac{|x_{channel}(t) - \mu_{user,channel}(t)|}{\max(\sigma_{user,channel}(t), \epsilon_{rate})}, 0, \text{cap}\right), \quad c_{channel} > 0 \quad (35)$$

In here, $c_{channel}$ represents a calibration value calculated for that specific communication channel. Equation (35) does not calculate this value. A baseline calculated separately for each channel already removes one part of the ordinary difference between the channels. Due to that reason, $c_{channel}$ should only include a remaining difference measured for that channel. This value needs data from the relevant communication channel. Hence, this paper does not assign a value it has not earned.

Equation (35) only gives a channel calibrated typing rate difference value. It does not change $TRD_{channel}(t)$ into the affective axis or hexarousal. The affective axis is a separate system process. It is a value which moves to higher levels as the user's typing remains in a sustained engagement. Equation (35) does not produce this movement by itself. A separate process needs to use the typing rate difference values across several keypresss before the system can make that statement. Professor Russell's circumplex model keeps "arousal" as the name of Professor Russell's own axis (refer [28]). That name does not replace the name of the IntentSpider signal.

This equation also does not claim one fixed behavior when the human moves between communication channels. It gives a warning that the system needs separate data for each channel. If one channel does not include a sufficient amount of earlier typing data, the system cannot state that its baseline for that channel is calibrated. In that state, the system can use a per user baseline which is dynamically calculated based on the huamn's typing patterns.

7. Necessity Gating Algorithm and Suggesting Functions

We previously mentioned that, processes from Section 3 to Section 6 select which next word's position and which net word to rank as first. But those processes do not mention the system should display a suggestion or not display a suggestion. A system which always displays its first ranked next word can display a word even when that word has a lower support value. This is a common functionality in modern keyboards as method, we believe, for giving the users an illusion of control over their data and fabricating the accuracy. Repeating this process can reduce the usefulness of the keyboard. Due to that reason, this section provides a gating process for that specific decision. This process is added after the ranking processes in Section 3 to Section 6 as well.

7.1 Using The SCAMPER Method to Generate This Process and Obtain Results

The process below was generated using SCAMPER, or Substitute, Combine, Adapt, Modify, Put another use, Eliminate, and Reverse. SCAMPER is a checklist algorithm, rather, a simple algorithm with simple logic gates. (refer [23]) built after American marketing executive Alex Osborn's research work. (refer [24]). This method can generate a system design. But the method is not proving a result, such as a 1 or 0. Due to that reason, Section 7.5 and Section 10.4 test this process based on its own results.

Naming this method in here is a deliberate difference from Section 3 using Section 6. Those sections build from cited and already established processes. After that, they adapt them into a different setting. This section starts from an acknowledged creativity method. Which means that, Section 7.6 keeps it under a stricter and not yet established level than the processes in Section 3 using Section 6.

- **Substitute.** The implicit rule which always displays the first ranked next word is replaced by a continuing necessity score value. The system compares that score with a margin value.
- **Combine.** The score combines signals which the architecture already produces. This includes the entropy confidence signal in Section 4.4 and the valence based urgency value from the seed context in Section 6.2. Which means that, this step does not create another input signal without an earlier source.
- **Adapt.** The exhaustion value in Section 3.2, Equation no. (14), becomes an input which reduces the necessity score. A pattern which was satisfied recently is less likely to pass the margin value again during its suppression period.
- **Modify and Magnify.** One possible extension changes the margin value for each human using the earlier data of that human, instead of using one population constant. This is the least decided part from this proposal. This section does not give a process for making that change.
- **Put to another use.** One considered use was inverting the similar score and using it to move an edge more faster than the ordinary process in Equation (3)/(8). This version does not use that extension. It adds another process without a required role in the current system.

- **Eliminate.** Another option removes the necessity gating process. And it lets the human ignore an incorrect suggestion. This remains as the null comparison B0 in Section 10.2. It is not removed from the evaluation process.
- **Reverse.** The reversed point is whether the system remaining without a suggestion gives an information to the human. This remains as an open question. Section 10 measures the quality of the displayed suggestions. It does not measure the human facing result of an absent suggestion. (refer Section 13)

7.2 The Proposed Process

Let w represent one next word at time t . The necessity score combines three values which the architecture already produces.

$$N(w, t) = \beta_1 \cdot (1 - H_{norm}(p)) + \beta_2 \cdot \max(0, \text{valence}_{urgency}(c_t)) - \beta_3 \cdot \text{exhaustion}(w, t) \quad (15)$$

Two parts of Equation no. (15) need a separate definition. This is because the earlier signals do not give those parts directly.

- **Let** S_k be the set of k next words used by the gating process, where $k > 1$. The diffusion values across S_k are normalized until their total value becomes 1. In Equation no. (15), p refers to that normalized distribution. In here, define $H_{norm}(p) = H(p)/\log(k)$. This is the entropy value from Section 4.4 divided by the entropy value of a uniform distribution across those k next words. Hence, $H_{norm}(p) \in [0, 1]$. And $1 - H_{norm}(p)$ gives a greater value when the diffusion result gives more support to a limited number of next words. If the total diffusion value across S_k is 0, the system does not execute Equation no. (15). And it does not display a suggestion. The entropy value $H(p)$ in Section 4.4 did not include this normalization. This section adds it because Equation no. (15) combines entropy with two inputs scaled from 0 to 1.
- **Define** $\text{valence}_{urgency}(c_t) = \max(0, -\text{val}'(c_t))$, where $\text{val}'(c_t) \in [-1, 1]$ is the typing frequency based context valence from Section 6.2, Equation no. (20). This gives a greater urgency value when the valence direction is more negative. And a valence value which is 0 or greater adds no urgency value. This use remains separate from the sign direction used by Equation no. (7) for the edge update. An earlier description only named this as a simple transform with values which are not negative values. This definition gives the exact transform used by Equation no. (15).

The $\text{exhaustion}(w, t)$ value comes from Equation no. (14) in Section 3.2. More than one edge can end at the similar next word w . Due to that reason, this section uses the greatest exhaustion value across the edges which end at w .

$$\text{exhaustion}(w, t) = \max_{(u, w) \in E} \text{exhaustion}((u, w), t)$$

This gives one value to Equation no. (15). And it prevents the gating process from ignoring an active exhaustion value only because another edge ends at the similar word. This aggregation is a design choice made in this section. Section 10.4 needs to test its result together with the other parts of the gating process.

β_1 , β_2 , and β_3 represent the three coefficients. κ_N represents the margin value. Similar to the other free parameters in this paper, their values need to

come from the experiments in Section 10.5. The other free parameters include θ_{TF} , η_{TF} , τ_v , τ_{seed} , η , α , ϵ , γ , ρ , τ_x , a , b , and Δt_{ref} . T and the earlier promotion margin value θ were removed in Section 3.3. This section does not assign values to β_1 , β_2 , β_3 , or κ_N without that data.

The notation uses κ_N instead of another τ or θ symbol. The τ name is already used by the decay constants τ_v , τ_0 , and τ_x . And the θ name was used by the promotion margin value in Equation (3)/(8). The coefficients use β_1 , β_2 , and β_3 instead of w_1 , w_2 , and w_3 . This paper already uses w for the edge coefficient values w_{stable} and $w_{volatile}$. Equation no. (12) also uses $w_i(\rho)$ for the support pool coefficient values. Reusing w for a third type of a value can connect three separate processes under a similar notation. Due to that reason, this section uses the β symbols.

The system displays a suggestion only when $N(w, t) > \kappa_N$. When the score is equal to or lower than the margin value, the system displays no suggestion.

The shock retreat process in Section 6.9.4 changes this margin value for a limited cool down window w_{shock} after a detected shock retreat event in Equation no. (30).

$$\kappa_N(t) = \kappa_N + \Delta\kappa_{shock} \cdot \mathbb{I}[t \text{ is within } w_{shock} \text{ of the most recent shock event}] \quad (36)$$

In here, $\Delta\kappa_{shock} > 0$ is another free parameter. The system uses $\kappa_N(t)$ instead of the ordinary κ_N value on every keypress. Outside a cool down window, $\kappa_N(t) = \kappa_N$. Which means that, the ordinary gating process remains unchanged outside that limited period.

Equation no. (36) also corrects the earlier direction. Equation no. (15) displays a suggestion when $N(w, t)$ is greater than the margin value. Lowering this margin value makes the requirement easier to pass. Which means that, the system can display more suggestions. A response which is more careful after the event needs a greater margin value. Due to that reason, Equation no. (36) adds $\Delta\kappa_{shock}$ instead of subtracting it.

7.3 Relationship With the History Based Task Scheduling Process and the Complete System

The History Based Task Scheduling process in Section 6.8 and the gating process in this section answer two separate points. And the system executes them in a sequence. Equation no. (13) decides which one ranks first across several active next words. Necessity gating decides whether that first ranked next word passes the requirement for being displayed. Which means that, one next word can become the first ranked word and after that, fail the necessity gating process.

Local push first produces the ranked next words. After that, the system executes Equation no. (15) for the first ranked next word before returning the result. This order has one specific reason. In Section 6.8.5, Equation no. (13) can move a newly active next word before another word which has a greater residue. That newly moved word has a lower amount of earlier information supporting it. Due to that reason, this is a specific state where the necessity gating process needs to check the result after the ranking process.

Another considered order executed $N(\cdot, t)$ for every active next word before Equation no. (13). After that, it would execute Equation no. (13) only for the

words which passed the gating process. This version does not use that order. In that process, the necessity score also becomes a second ranking signal. And it would combine the result of Hypothesis 5 about gating with the result of Hypothesis 4 about the History Based Task Scheduling process in Section 10.4.

Updated Algorithm 1 With Added Theories

Required inputs and current state - the selection event (u, v, t) , the context window c_t , the typing information needed to calculate $TRD(t)$, the current edge set E , where each edge carries its current values of w , TF , and s , the current suppressed-edge set $E_{\text{suppressed}}$, and the stored shock time, if one exists.

Notice - iff - if and only if, = - equal, <= - less than or equal to, >= - greater than or equal to, === or == - equality

1. Calculate val' by applying ScoreValence to the context window c_t , as defined in Equation (20) and Section 6.2.
Notice - This calculation is not changed compared to the first version.
2. Calculate the standardized typing rate difference $TRD(t)$ using Equation (23), as described in Section 6.9.1. This value provides the typing input for the affective axis, this is also called hexarousal.
3. Update the sign value $s(u, v, t)$ as described in Section 6.3.
Notice - This calculation is not changed compared to the first version.
4. Update the edge weight $w(u, v, t)$ using the first equation. (equation no. (1))
Notice - This calculation is not changed compared to the first version.
5. Update the currently active set of prey $P(t)$ using the logic in equation (24) as described in Section 6.9.2.
6. If $P(t)$'s active prey items > 1 , calculate the budget for reinforcement $B(t)$ using the equation no. (25).
7. Distribute the budget for reinforcement processes to nodes or members of $P(t)$ (the equation (26))
8. Update the transmission fidelity value $TF(u, v, t)$ using equation no. (16), as described in Section 3.3.
Notice - The update uses the outcome of the currently existing system unit, Which is $outcome(u, v, t)$.
9. If $TF(u, v, t)$ is greater than θ_{TF} , then mark (u, v) as a transmission type edge and include it in $W_{\text{transmission}}$, which was denoted in the equation no. (17).
10. else - mark (u, v) as a capture type edge and remove it from the process of adapting.
11. End and return the transmission type check.
12. Check whether the ordinary suppression condition is in the state of true for the history of $s(u, v, \cdot)$, as described in Section 6.4.
13. If the condition is true, move (u, v) into $E_{\text{suppressed}}$.
Notice - This ordinary suppression check is a separate process from the transmission type check. Avoid confusion. It is not the same.
14. End the ordinary suppression check.
15. As denoted in the section 6.9, then calculate $shock(t)$ using the equation no (30)
16. If $shock(t) == 1$, then within a short time, move the specific edge (u, v) assigned to that event into $E_{\text{suppressed}}$ (refer equation (31))

17. If shock suppressing function is triggered and executed, store and record the time information. (This recorded time can be used later for other purposes.)
18. Next keypress, use the last k typed words to start the predicting process.
19. Update the streak length $\ell(t)$ using the threshold θ_D , as described in Section 6.9.3.
20. Then calculate the $\zeta(\ell(t))$ using the equation (28)
21. Calculate the effective adapting parameter $\alpha_{\text{effective}}(t)$ using the equation no. (29)
22. Create the seed stage vector $s(0)$ by applying FanDispersionSeed to $u_t, \dots, u_{t-k+1}$ using the methods described in the subsection 4.1.1.
Notice - This calculation is not changed compared to the first version.
23. Calculate $\hat{p}$ using LocalPush to $s(0)$, W_{signed} , $\alpha_{\text{effective}}(t)$, and ϵ .
Notice - Please notice that we use $\alpha_{\text{effective}}(t)$ in the position of the fixed value α .
24. Then rank the words by their relevant value in $\hat{p}$ using the equation (13), (subsection 6.8.5) then it should store the three highest ranked words in the "nextwords"
25. the assign the value of w^* to the word, which is having the rank number 1 in the nextwords.
26. the process of calculating the novelty value $N(w^*, t)$. (equation (15).)
27. Generate the time depending novelty threshold value $\kappa_N(t)$, (refer section 7.3 and equation no. (36))
28. Use the ordinary threshold κ_N iff, record shock= 0 inside the shock window w_{shock} . else if a specific shock is inside that window, use $\kappa_N + \Delta\kappa_{\text{shock}}$.
29. If $N(w^*, t) \leq \kappa_N(t)$, return the empty set, then return and stop the algorithm. In this case, suggestions are not displayed.
30. Otherwise, return nextWords and $H(\hat{p})$, in cases where $H(\hat{p})$ is the uncertainty signal, which was described in section 4.4.

The added steps calculate the TRD(t) value, apply the word group reinforcement, detecting the event of shock, and it changes the diffusion radius value. And the gating process uses the temporary margin value in the equation no. (36) instead of one constant margin value.

The settling event in Section 6.9.5 does not appear as a keypress step in this algorithm. It starts after T_{grace} passes without another qualifying activation across the edges in $P(t)$. Which means that, it is not an event created by one keypress. The implementation can run it as a separate check during an inactive period. The exhaustion decay in equation no. (14) is also the same. And because of that, the settling check does not need to become a branch inside the per keypress process above.

8. Privacy Related Processes and Disadvantages

8.1 Processing the System Data Inside a Single Device

The main advantage of this architecture in terms of privacy comes from processing the system data inside a single device. The graph and the earlier ordinary typing speed value used for Δt_{ref} are processed in that device. (refer Section 6.2) The update processes in Section 3, Section 4, Section 6 and Section 7 are also processed inside the device.

IntentSpider does not use an external server component for these processes. As mentioned earlier, the keypresss, the gradient values and the values calculated from the graph are not transmitted to an external server or another external component in order for the IntentSpider system to function. Which means that, this privacy advantage comes from where the data is processed and stored.

This privacy advantage is limited to the data being processed inside one device. It is different from the mathematical process of differential privacy. (refer [6]) Differential privacy uses separate calculations to limit what can be found from stored information. The process described in this section does not give that similar result when another human reads or copies the data stored in that device.

If the IntentSpider system shares the graph state between several devices through a cloud server, then the data does not remain inside a single device. In that state, the system requires a separate privacy process for the data transmitted between those devices.

8.2 The Differential Privacy Process and the Research Questions

IntentSpider does not include a separate and verified differential privacy process in this architecture. Applying differential privacy to a signed and function typed graph creates two research questions. The first one is adding a calibrated noise value to a graph which has a non Euclidean structure. That noise value requires a mathematical process which verifies the privacy result it produces.

The second question is related to the information accumulated in the graph across a longer period. One stored data object and a general personal characteristic are different types of information. Which means that, limiting what can be found from one stored data object does not directly limit what can be found from the accumulated graph structure. Information such as the gender of the human and another personal characteristic should not be identified and used as a system feature from this graph.

These two questions need separate research processes. Section 12 discusses them as limitations and future research points. Due to that reason, IntentSpider does not use a privacy equation which has not been verified for this graph.

8.3 Identification Through the Typing Interval Pattern Values

Section 6.2 changes the word list based valence scoring process into the typing interval pattern based process. From this process, it does not need to read the content of a word for calculating $val'(c_t)$. This gives a one advantage in privacy. But it also adds a separate disadvantage.

keypress dynamics research uses the time between the keypresss and the correction rate for identifying and safely proceeding with auth. (refer [27]) The

Δt_{avg} , Δt_{ref} and κ values used by IntentSpider include the similar type of typing information. These values are processed inside the device and they are not transmitted to an external server. Due to that, this does not add another transmission disadvantage.

But if another human reads or copies the stored data of that device, these values can show the specific way how that human types. The graph shows what that human connects with the other words. The typing interval pattern values show how that human uses the keyboard. Specifically, today with Artificial Intelligence and advanced reverse engineering tools, this is a real possibility. And these are two separate types of personal information.

The Δt_{ref} value and the other per user timing values need the similar stored data protection used for the graph. This is because these values include the human typing patterns. Which can be classified as highly sensitive Personally Identifiable Information, or PIIs.

8.4 Protection and Safe Storage of IntentSpider Graph Data

The graph described in Section 3, Section 6 and Section 7 includes the private word connections of a human. From these word connections, information related to health conditions, relationships and the situational vulnerabilities can be identified. This is depending on what that human types and the specific time when those words are typed.

The graph data requires protection while it is stored. This includes an encrypted data storage process. And also, the cloud storage should not be enabled as the default process. The IntentSpider system should allocate the human a direct access setting for inspecting and deleting selected graph parts. This protection is required even though the graph is not transmitted during the normal operation of IntentSpider.

8.5 About Typing Rate Values and Typing Sub States

The μ_{user} and σ_{user} values in Section 6.9.1 are calculated from the earlier typing rate of that person. These two values are another set of typing based personal information. They are processed inside the device. But if the stored data from that device is obtained by another person (examples include data theft, legal asset freezing, malware), those values are available together with Δt_{ref} and the graph. Due to that reason, the μ_{user} and σ_{user} values need the similar stored data protection.

In here, the TRD(t) is a variable which does not have a positive or a negative direction. (TRD(t) is a scalar quantity) The discovered per user typing sub states in the subsection 6.9.6 include several typing patterns instead of a one value.

These can include a faster typing state, slower typing state, a different correction rate state and a pause pattern state. Even though the grouping methods and states are not equal to the real biological emotional states of the human, it can create a complete profile from the measured typing behavior, into a certain extent. This profile includes more personal typing information than one timing value. Hence, the inspection and deletion process needs to include the discovered typing sub state groups separately. It should not be limited only to that graph. s

9. The Processing and Storage Amounts Required Inside a Single Device

Due to the variances in modern Operating Systems in Tablet, Mobile and Web Interfaces, this section outputs the approximate calculation values for the storage and the processing amounts. They are calculations made under the assumptions denoted below. These calculations provide information to understand whether the system is possible inside a mobile device into a some extent.

9.1 Storage Requirements

Suppose a human who types heavily builds approx. $|V| \approx 5 \times 10^4$ number of different words within the timeframe of several years. And suppose the average node degree is $d \approx 20$. The earlier calculation used d across the two combined channels. But Section 3.2 now uses a one single edge set. So value d now outputs the degree for each node inside that one edge set.

Each edge stores a signed coefficient, a raw coefficient, a transmission fidelity value, and the time when it was used finally. Each one uses 4 bytes. Which means those four values use 16 bytes for a one edge (a word connection). In addition to that, the adjacency structure can use approx. 8 bytes for the pointers or the index values. Hence, one edge uses approx. 24 bytes.

$$|V| \times d \times 24 \text{ bytes} \approx 5 \times 10^4 \times 20 \times 24 \approx 24 \text{ MB.}$$

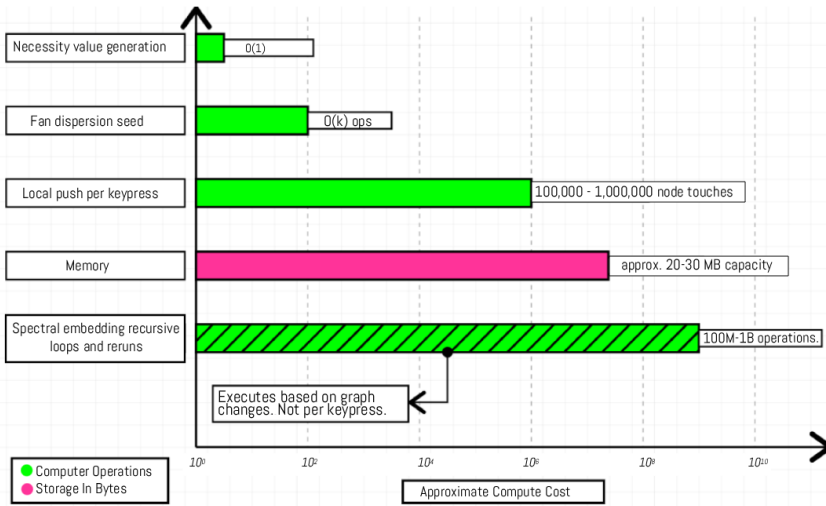


Figure 11. Approximate storage and processing cost components of the IntentSpider system based on assumptions in section no (9)

The frequency based valence signal in Section 6.2 replaces the earlier word list. That word list was a old process with a few thousand entries. And it used an amount lower than 1 MB. The new process keeps several values for each human. This includes Δt_{ref} and a short rolling window of the recent Δt values and the backspace counts. Which means that, this amount is small even when compared to the word list it replaces.

The two amounts are small when compared to the the storage and RAM amounts in current mobile phones. Researche articles featured on an Internet Website Technobaboy.com and Counterpointresearch.com in 2025-2026 denoted that the average RAM of a mobile device has reached to 8.4 Gigabytes and General Storage to 279 Gigabytes. (refer [34], [35]) In addition to that, this version of IntentSpider did not include or measure the manual data sweeping values, storage allocation, garbage collection, etc.

Section 6.9 adds several more values for each human. This includes μ_{user} and σ_{user} for the typing rate. And it includes $\mu_{val,user}$ and $\sigma_{val,user}$ for the valence value. In addition to that, it includes the current sustained period length $\ell(t)$ and a shock time value. Each edge included in the shock process also keeps a shock state and a decay clock (refer Section 6.9.4). These values use an amount within tens of bytes together. Meaning, their amount is small in the similar way as Δt_{ref} .

The discovered typing sub state groups in Section 6.9.6 need a separate note. Storing k discovered center values across the five dimensional feature vector in Equation (33) is also a small amount. But the online grouping process which discovers those center values is a continuing background processing process. Section 6.9.6 does not decide one clustering algorithm. Due to that reason, this estimate does not output a processing limit for that part.

9.2 The Processing Time for Each Keyboard Input (character addition, backspace, delete, other actions, formatting, etc.)

The local push process in Section 4.3 creates the main processing amount for each keypress. In a graph with coefficients which are positive values and a fixed edge set, this process has the $O(1/(\alpha\epsilon))$ node touching limit. For the ordinary Personalized PageRank values $\alpha \approx 0.15$ and $\epsilon \approx 10^{-5}$, this limit outputs approx. 10^5 to 10^6 node touch operations in a greater case. Each node touch uses several float values. This is approx. to be an amount lower than 1ms (one millisecond) or within a small number of milliseconds on a normal mobile CPU. But this is a extension from the processing limit. This is not a measured result. In addition to that, a personal graph with a large delta difference in between the degrees of that node may be different from this ordinary case.

Building the fan dispersion starting point from Equation (19) uses $O(k)$ work for k recent words. It also requires one dispersion value lookup for each word. This amount is small when it is compared to the local push process.

The necessity value from Equation (15) in Section 7.3 adds another small amount. This process outputs a fixed coefficient based addition of three values which are already calculated for each next word. Which means that, it uses time complexity Big $O(1)$ processing for each next word. So, due to that reason, there will be no changes to the processing order denoted above.

Two processes from Section 6.9 change this calculation in two separate ways. First, the shared cluster reinforcement in Equation (26) and the shock detection in Equation (30) use $O(1)$ or $O(|P(t)|)$ processing for each keypress. The $|P(t)|$ value is limited by the number of next words which can remain active at that time. This number can be small in a practical system. So these two processes add a small processing amount to the overall RAM usage.

The growing radius process in Section 6.9.3 is different. The $O(1/(\alpha\epsilon))$ limit is no longer calculated with a fixed α value. The earlier 10^5 to 10^6 node touch cal-

ulation uses $\alpha \approx 0.15$. Once $\alpha_{effective}(t)$ reduces during a sustained typing rate difference period, that earlier calculation becomes a lower processing amount and not the ordinary processing amount for that period.

When $\alpha_{effective}$ reduces to 0 gradually, the processing amount from that same limit increases without a upper limit. This is the processing disadvantage. This disadvantage was already mentioned in the subsection 6.9.3 and Section 4.3. In here, it becomes a direct processing time disadvantage. And this function output displays during the active typing periods. Which means even though this process should be working at that timeframe, it can be crash the software due to this reason.

Whether $\alpha_{effective}$ needs a minimum value is not decided in this paper. A minimum value can keep the greater processing time inside a usable amount. But it also limits the maximum diffusion can travel during a hexarousal period (a consistant period). Due to that reason, this remains as an implementation question.

9.3 Comparing Against Already Existing Processes

This below table (2), which is a feature based comparison between the IntentSpider Webnet and two other methods which were mentioned in Section 2. And it includes the related external research systems. (These systems are discussed in detail in the main section 2.). The reason this table does not include a separate necessity gating column is that, none of these other external systems possess a system to gate next word suggestions based on a theoretical confidence value.

Table no. 2. A comparison between the already existing external processes and the IntentSpider Webnet.

Computer Arch.	Inside a device or without an external server?	Supports the starting state without earlier data?	Supports valence?
n gram architecture (SwiftKey (old version), Google GBoard) (refer [14,17])	Yes	No	No
Federated RNN keyboard (refer [10])	Only for gradient values	Weak	No
DualNet, CLS ER, and ITDMS (refer [21,2,22])	Not Applicable	Not Applicable	No
IntentSpider Webnet V1	Yes	Yes	Yes (mentioned in main section 6.)

9.4 The Processing Amount of the Periodic Graph Embedding Recalculation

Calculating all exact eigenvectors directly from a $|V| \times |V|$ Laplacian is not possible for the graph size assumed in the storage calculation. In that calculation, $|V| \approx 5 \times 10^4$. A more direct and applied simplifying can use $O(|V|^3)$ processing. Which means it uses an amount approx. 100,000,000,000,000 operations

(more than 100 Trillion Ops per calculation). This amount is outside a processing amount available inside a general mobile device such as an iPhone.

But Equation (37) needs only the two leading eigenvectors. Iterative processes such as the Lanczos iteration can calculate these two values from the sparse structure of the graph. Another option is the power iteration process on a shifted or inverted operator. The approximate processing amount is $O(k \cdot \text{nnz}(L) \cdot \text{number of iterations})$. In here, $k = 2$. And $\text{nnz}(L) \approx |V| \times d \approx 5 \times 10^4 \times 20 \approx 10^6$ entries with values which are not 0. The number of iterations for the first two eigenvectors can be from tens to a lower number of hundreds. This is depending on the difference between λ_2 and λ_3 .

These values output approx. 10^8 to 10^9 (100 million Ops to 1 billion Ops) basic operations for each recalculation. This is more higher than one local push call from Equation (6). But this process runs in regular intervals and not on every single keypress. Due to that reason, it can remain lower than one second on a current mobile CPU. This is also an approximate calculation.

Table 5. Hypothesis families fixed before execution. The Phase 0 hard gate applies before any statistical verdict is assigned.

ID	Question	Comparison	Designated evidence
H1	Does diffusion earn its complexity in cold-start prediction?	B4 against B1	Difference between the M2 and M1 improvements
H2	Does valence-gated signed diffusion solve valence conflict?	B4 against B2	M3 suppression correctness, with M1 non-regression
H3	Does B4 remain competitive with an adjacent published architecture?	B4 against B3	M2 and M3 accuracy, with a pre-declared non-inferiority margin
H4	Does residue-priority arbitration help without trading away cold-start behavior?	B5 against B4	M3, M1, and M2, reported through the four-cell outcome rule
H5	Does necessity gating reduce unhelpful suggestions while preserving shown-suggestion quality?	B4G against B0	M4, conditional M1, and M10
H6	Does the emotional engine help sustained engagement?	B6 against B5	M5 with M1/M2 non-regression; M6 correctness and false positives separately
H7	Does high fan dispersion predict a position nearer the embedding centroid?	One graph at a time	M7 Spearman correlation with a permutation null
H8	Does a shared low- ϕ token create tighter reconvergence?	Low- ϕ against high- ϕ pairs	M8 reconvergence-gap ratio, stratified before comparison

10. Experiments and Validation

10.1 Evaluation Questions and Experimental Structure

The evaluation was organized around the eight falsifiable questions produced by the architecture. H1 compares the function-typed diffusion system with a Kneser-Ney smoothed trigram on the cold-start condition introduced in Section 4.4. H2 isolates the signed valence extension of Section 6. H3 compares the proposed system with an adapted dual-memory continual learner and remains the most important external comparison, because the current transmission-fidelity architecture does not share the dual-memory baseline’s mechanism. H4 evaluates residue-priority arbitration. H5 evaluates the independent decision of whether a suggestion should be displayed. H6 evaluates the bundled hexarousal processes on the sustained-engagement cases they were built to address. H7 and H8 evaluate the geometric claims of Section 4.5 without treating the embedding as another prediction architecture.

The system registry fixed the comparison arms before execution. B1 is the Kneser-Ney smoothed trigram. B2 is unsigned function-typed diffusion. B3 is

the adapted dual-memory continual learner. B4 is the proposed core system with transmission-fidelity role assignment, fan-dispersion seeding, signed diffusion, and the suppressive set. B5 adds residue-priority arbitration. B6 adds compounding reinforcement, growing engagement radius, shock-retreat, and grace-period settling to B5. B0 always displays B4’s top-ranked candidate, while B4G activates the necessity gate. As specified in Section 4.5, no B7 was created for the geometric lens, because the lens observes a graph and does not change prediction, reinforcement, suppression, or gating.

The complete protocol contained 56 experiment identifiers. E01–E07 formed the instrumentation and validity gate. E08–E16 covered baseline and system execution. E17–E24 were the eight hypothesis tests. E25–E40 contained the eleven named ablations (a)–(k) and their parameter sweeps. E41–E56 converted the paper’s implementation questions into convergence, cost, connectivity, embedding, double-cold-start, valence-proxy, scope, interaction, generalization, and feasibility probes. The absence of an ablation (l) was deliberate: H7 and H8 are direct geometric measurements, not removable behavior-producing mechanisms.

10.2 Data, Splits, Metrics, and Statistical Rules

The protocol defined two data sources. D1 is a synthetic personal typing history in which the required event labels are known by construction. Its seven conditions are: a stable base stream (D1-A), valence-conflict bursts (D1-B), nearby low-relevance distractors (D1-C), sustained typing-rate-difference bursts (D1-D), shock and frustrated-typing controls (D1-E), confirmed and lapsed settling sequences (D1-F), and diverge-then-reconverge sentence pairs (D1-G). D2 is the per-participant real personal corpus path. Its privacy rule follows the architecture: each corpus remains on the participant’s device or in that participant’s isolated offline analysis, and participant results are never pooled.

The split rule was chronological. The first 90% of each stream was assigned to training and the final 10% to held-out evaluation. The cold-start subset contained held-out targets appearing for the first time in the held-out portion while graph neighbours already possessed transmission-type edges. The protocol also separated the valence-conflict, sustained-engagement, distractor-labelled, genuine-shock, frustrated-typing-control, and reconvergence subsets. This separation prevents a general accuracy number from being used as a proxy for a mechanism-specific claim.

Ten metrics were pre-defined: overall top-3 accuracy (M1), cold-start top-3 accuracy (M2), valence-conflict accuracy and suppression correctness (M3), unhelpful-suggestion rate (M4), sustained-engagement top-3 accuracy (M5), shock-suppression correctness and false-positive rate (M6), centrality-dispersion correlation (M7), reconvergence-gap ratio (M8), latency and instrumentation cost (M9), and suggestion-shown rate (M10). M4 is always interpreted with M10, because a gate that never displays a suggestion would otherwise minimize the unhelpful-suggestion rate trivially. Similarly, M6 correctness is never collapsed with its frustrated-typing false-positive rate, and the low- ϕ and high- ϕ strata in M8 are never pooled.

The planned comparisons used 10,000 paired bootstrap resamples for accuracy and rate differences, McNemar’s exact test for paired suppression outcomes, and Spearman rank correlation with a 10,000-permutation null for M7.

Table 6. Mechanism-to-code correspondence established by E01.

Mechanism	Implementation location	Primary symbol	Status
Transmission-fidelity role assignment	src/graph.cpp	updateTF	Located
Fan-dispersion seeding	src/engine.cpp	predict	Located
Signed diffusion read-out	src/diffusion.cpp	localPush	Located
Suppressive set	src/graph.cpp	reinforce	Located
Residue-priority arbitration	src/engine.cpp	predict	Located
Necessity gate	src/engine.cpp	predict	Located
Compounding reinforcement	src/engine.cpp	selectionEvent	Located
Growing engagement radius	src/arousal.cpp	alphaEffective	Located
Shock-retreat	src/valence.cpp	shock	Located
Grace-period settling	src/engine.cpp	tick	Located
Spectral embedding	src/embedding.cpp	compute	Located
Fan dispersion $\phi(u)$	src/graph.cpp	fanDispersion	Located

The significance level was fixed at 0.05. Five random seeds were required for each reportable configuration. Holm correction was specified within each hypothesis family, while H1-H8 remained separate claim families. These statistical procedures belong to E17-E24 and are not applied to records excluded by the Phase 0 gate.

10.3 Implementation-Fidelity Audit

E01 traced the paper mechanisms to executable symbols in the current implementation. All 12 mechanisms required by the audit were located with high confidence. This result is important because it establishes that the evaluated repository is an implementation of the architecture described here rather than a placeholder carrying only the IntentSpider name.

The wider repository audit also compiled the current version1/ and legacy C++ paths using the C++17 toolchain and identified the WebAssembly C interface used for per-keypress integration. The current engine is organized around the graph, diffusion, valence, arousal, arbitration, and embedding components. This explicit module structure supplied the traceability required for E01.

10.4 Artifact Integrity and Reproducibility Boundary

The delivered result package contains a campaign status report, detailed result and blocker reports, a deviations record, a reproduction guide, the Phase 0 and full experiment registries, the hypothesis registry, and the validated evidence files. A checksum registry records SHA-256 values and row counts for the final reports and Phase 0 CSV files. The mechanism audit, determinism comparisons,

generator checks, split record, metric tests, cadence diagnostic, retired-symbol check, and parameter snapshot are therefore traceable to named immutable artifacts and to one repository commit.

The campaign also recorded two reporting decisions that are material to interpretation. Placeholder outputs that contradicted the clean rebuild were rejected rather than merged into the final evidence. Premature B0-B6 records from the clean rebuild were preserved rather than deleted, but were excluded from the scientific result registry. These decisions keep the engineering history inspectable while ensuring that only evidence which passed its declared dependency path is used in the paper.

11. Conclusion

The function typed and diffusion based architecture is the main system described in the IntentSpider system for the on device next word prediction feature. In this system, every edge receives a position from a current ongoing transmission fidelity value. This value can increase or reduce in all of those two directions. Which means that, the relevant position of an edge is not decided by a one way consolidation clock, but rather, it is decided based on whether that edge has a reliable ability to predict the next word which the human may select. In addition to that, the initial seed values for the diffusion process are not created using only the single most recent word. Those values are created using a combination of several recent individual words. And the coefficients are based on the scattered distribution of those word connections. (refer Section 3 and Section 4)

The IntentSpider system is novel due to the reason that, even though it is based on already existing research, it combines those research processes into a different system. (detailed explanations are in Section 2) For an example, the on device text prediction, two memory system processes, local computation, graph diffusion process, reinforcement, signed graph methods, typing interval pattern values and spectral graph embedding are already existing research areas into a some extent. But this system gives graph edges a function type from a current ongoing and freely reversible transmission fidelity value. And also, this system reads that graph through diffusion with the recent word combination described above. These above information are given in order to identify the novel contributions made by the IntentSpider system and the already existing research used by the system.

In the Section 5, the raw coefficient update and the transmission fidelity update cannot separate a next word selection which confirms the intent, from a next word selection which ends the started intent. In the Section 6, it provides the signed and valence gated extension for this problem. The valence signal in this version comes from the typing interval patterns and the correction rate. It is not derived from the meaning of a specific word list. But the result is depending on whether those typing pattern values can separate those two selection types in a mobile application, a desktop application, command line interface, etc. It also gives the necessity gating process in the Section 7. This is a separate process for deciding whether a suggestion should be displayed in the output or should not be displayed. However, it does not decide the next word which is ranked by Section 3 and Section 4 for the first rank. And this necessity gating process is a hypothesis which is more experimental than the function typed core. Due to that reason, the predicting architecture and the output displaying decision process should not be confused with each other.

In the Section 6.9, it introduces a second affective axis in addition to the valence signal. And Professor James Russell's research paper uses the term "arousal" for that axis. The IntentSpider system uses a separate term for denoting its own typing pattern and behavior based axis as the "hexarousal axis". After that, another six processes are built on top of this affective axis. This includes the shared cluster reinforcement process, the growing engagement radius, the sudden negative selection response, the settling event, the discovered typing sub states and the behavioral interaction coefficient. Those six processes came from the motivation by an attack and struggle analogy, which was

described in the introduction section. As stated in the research papers from Professor James Russell and Professor Joseph Walther (refer [28], [29]), those researches show the background for the two affective axes or dimensions and the communication channel difference. But those research papers are not directly relevant to these six processes mentioned above. Which means that, these theories are more experimental as when considered as system components. Also, it includes the highest number of parameters which are not proven using mathematical proofs. In addition to that, the section 4.5 adds a system component which produces a live 2D spectral embedding to that same graph. Meaning that, every typed sentence transforms into a path through a plane. And the geometrical difference between these two paths results in a geometric shape which is also measurable. The IntentSpider system consists of two experimental hypotheses from this live 2D spectral embedding. But they are testable. This component does not change the system functions or operations. It can be considered as a optional, developer focused deep debugging tool instead. Due to that, the spectral embedding is not used as an input for the prediction process. It is also not used for the reinforcement process, or the necessity gating process as well.

References

- [1] R. Andersen, F. Chung, and K. Lang. Local graph partitioning using PageRank vectors. In *47th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2006)*, pages 475–486, 2006.
- [2] E. Arani, F. Sarfraz, and B. Zonooz. Learning fast, learning slow: A general continual learning method based on complementary learning system. In *International Conference on Learning Representations (ICLR)*, 2022.
- [3] D. Cartwright and F. Harary. Structural balance: A generalization of Heider’s theory. *Psychological Review*, 63(5):277–293, 1956.
- [4] F. Chung. The heat kernel as the PageRank of a graph. *Proceedings of the National Academy of Sciences*, 104(50):19735–19740, 2007.
- [5] M. Dorigo. *Optimization, Learning and Natural Algorithms*. PhD thesis, Politecnico di Milano, 1992.
- [6] C. Dwork. Differential privacy. In *33rd International Colloquium on Automata, Languages and Programming (ICALP)*, pages 1–12, 2006.
- [7] C. Genest, S. Weerahandi, and J. V. Zidek. Aggregating opinions through logarithmic pooling. *Theory and Decision*, 17(1):61–70, 1984.
- [8] C. Genest and J. V. Zidek. Combining probability distributions: A critique and an annotated bibliography. *Statistical Science*, 1(1):114–135, 1986.
- [9] F. Glover. Future paths for integer programming and links to artificial intelligence. *Computers & Operations Research*, 13(5):533–549, 1986.
- [10] A. Hard, K. Rao, R. Mathews, S. Ramaswamy, F. Beaufays, S. Augenstein, H. Eichner, C. Kiddon, and D. Ramage. Federated learning for mobile keyboard prediction. *arXiv preprint arXiv:1811.03604*, 2018.
- [11] G. E. Hinton and D. C. Plaut. Using fast weights to deblur old memories. In *Proceedings of the Ninth Annual Conference of the Cognitive Science Society*, pages 177–186. Erlbaum, 1987.
- [12] C. J. Hutto and E. Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Eighth International AAAI Conference on Weblogs and Social Media (ICWSM)*, 2014.
- [13] G. Jeh and J. Widom. Scaling personalized web search. In *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pages 271–279, 2003.
- [14] S. Katz. Estimation of probabilities from sparse data for the language model component of a speech recognizer. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 35(3):400–401, 1987.
- [15] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [16] J. Klicpera, A. Bojchevski, and S. Günnemann. Predict then propagate: Graph neural networks meet personalized PageRank. In *International Conference on Learning Representations (ICLR)*, 2019.
- [17] R. Kneser and H. Ney. Improved backing-off for M-gram language modeling. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)*, volume 1, pages 181–184, 1995.
- [18] J. Kunegis, S. Schmidt, A. Lommatzsch, J. Lerner, E. W. De Luca, and S. Albayrak. Spectral analysis of signed graphs for clustering, prediction and

visualization. In *Proceedings of the 2010 SIAM International Conference on Data Mining (SDM)*, pages 559–570, 2010.

[19] J. L. McClelland, B. L. McNaughton, and R. C. O'Reilly. Why there are complementary learning systems in the hippocampus and neocortex: Insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3):419–457, 1995.

[20] L. Page, S. Brin, R. Motwani, and T. Winograd. The PageRank citation ranking: Bringing order to the web. Technical report, Stanford InfoLab, 1998.

[21] Q. Pham, C. Liu, and S. Hoi. DualNet: Continual learning, fast and slow. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021.

[22] R. Wu, K. Huang, H. Zhang, Q. Liu, G. Yu, J. Deng, and F. Ye. Information-theoretic dual memory system for continual learning. *Knowledge-Based Systems*, 325:113913, 2025.

[23] R. Eberle. *SCAMPER: Games for Imagination Development*. D.O.K. Publishers, 1971.

[24] A. F. Osborn. *Applied Imagination: Principles and Procedures of Creative Problem-Solving*. Charles Scribner's Sons, 1953 (3rd rev. ed., 1963).

[25] R. F. Foelix. *Biology of Spiders*, 3rd edition. Oxford University Press, 2011.

[26] R. Hergenröder and F. G. Barth. The release of attack and escape behavior by vibratory stimuli in a wandering spider (*Cupiennius salei* Keys.). *Journal of Comparative Physiology A*, 152:347–358, 1983.

[27] F. Monrose and A. D. Rubin. Keystroke dynamics as a biometric for authentication. *Future Generation Computer Systems*, 16(4):351–359, 2000.

[28] J. A. Russell. A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6):1161–1178, 1980.

[29] J. B. Walther. Computer-mediated communication: Impersonal, interpersonal, and hyperpersonal interaction. *Communication Research*, 23(1):3–43, 1996.

[30] M. Fiedler. Algebraic connectivity of graphs. *Czechoslovak Mathematical Journal*, 23(2):298–305, 1973.

[31] M. Belkin and P. Niyogi. Laplacian eigenmaps for dimensionality reduction and data representation. *Neural Computation*, 15(6):1373–1396, 2003.

[32] J. Duprez, M. Stokkermans, L. Drijvers, and M. X. Cohen. Synchronization between keyboard typing and neural oscillations. *Journal of Cognitive Neuroscience*, 33(5):887–901, 2021.

[33] F. Richlan, J. Schubert, R. Mayer, F. Hutzler, and M. Kronbichler. Action video gaming and the brain: fMRI effects without behavioral effects in visual and verbal cognitive tasks. *Brain and Behavior*, 8(1):e00877, 2018.

[34] M. S. Hwang, Y. Wang, and S. Bai. 2026 smartphone shipment forecasts revised down as memory shortage drives BoM costs up. *Counterpoint Research*, December 15, 2025.

[35] Omdia. Global average smartphone DRAM and NAND capacity climbs in 2026. *Technobaboy*, July 14, 2026.